SWIFTUI

CHEAT SHEETS

A Complete Reference for Modern SwiftUI

Views · Modifiers · Layout · @State · @Observable · Lists · Navigation · Sheets · SwiftData · Animation · Liquid Glass · Accessibility

59 REFERENCE SHEETS

CONTENTS

FOUNDATIONS

01 Introduction & Setup
05 Views & Modifiers
09 Layout
14 State & Data Flow
20 Controls & Input

DATA & NAVIGATION

26 Lists & Scrolling
30 Navigation
34 Presentation
37 Async Work & Lifecycle
40 Persistence

POLISH & PLATFORMS

43 Animation & Graphics
47 Styling & Customization
50 UIKit & Other Platforms
53 Quality
56 Architecture & Testing
58 Quick Reference

Covers iOS 17+ · Observation · SwiftData · Liquid Glass (2026)

Begin → Sheet 01

Table of Contents

Introduction to SwiftUI

WHAT SWIFTUI IS

SwiftUI is Apple's **declarative** UI framework, introduced in 2019. Instead of creating views and then changing them step by step, you describe what the screen should look like **for the current state**, and SwiftUI updates the screen whenever that state changes. Views are lightweight Swift structs, and one codebase runs on iOS, iPadOS, macOS, watchOS, tvOS and visionOS.

NOTES

- Views are values: cheap structs recreated on every update
- State lives in property wrappers (`@State`) and models (`@Observable`)
- UIKit and AppKit still run underneath; you can mix them
- Requires Swift; see the Swift set for language features

A COMPLETE SWIFTUI VIEW

```
import SwiftUI

struct CounterView: View {
    @State private var count = 0                // source of truth

    var body: some View {                       // describes the UI
        VStack(spacing: 12) {
            Text("Count: \(count)")
                .font(.largeTitle)
            Button("Add One") { count += 1 }  // change state…
                .buttonStyle(.borderedProminent)
        }                                       // …SwiftUI redraws
        .padding()
    }
}

#Preview { CounterView() }
```

CODE EXPLAINED

- → `View` — protocol every screen piece conforms to
- → `@State` — value SwiftUI stores and watches
- → `body` — computed again when state changes
- → modifiers — `.font` , `.padding` return new views
- → `#Preview` — live canvas in Xcode

MINIMUM OS

- SwiftUI: iOS 13 / macOS 10.15 (2019)
- This set targets iOS 17 / macOS 14 as a baseline

IMPERATIVE VS DECLARATIVE

UIKIT (IMPERATIVE)	SWIFTUI (DECLARATIVE)
`label.text = "3"`	`Text("\(count)")`
`addSubview / constraints`	`VStack / HStack`
`delegate callbacks`	`closures + bindings`
`reloadData()`	`state change redraws`

UIKIT / REACT EQUIVALENT

- **`UIKit`** View controllers + views you mutate imperatively
- **`React Native`** Function components + `useState` : the closest model

PLATFORMS

PLATFORM	NOTES
`iOS / iPadOS`	Primary target
`macOS`	Native Mac apps
`watchOS`	SwiftUI is the main UI API
`visionOS`	Windows, volumes, spaces

⚠ COMMON MISTAKES

- Trying to keep references to views and change them later: describe state instead
- Putting heavy work in `body` : it runs often
- Learning from 2019–2021 tutorials: navigation and state APIs have changed

✓ TIPS & BEST PRACTICES

- Think "UI = f(state)": change the data, not the view
- Use the preview canvas constantly while building
- Read Apple's SwiftUI tutorials alongside this set

QUICK SUMMARY

- `View` structs describe the UI
- `@State` changes trigger redraws
- modifiers build up appearance

App Structure

APPS, SCENES & WINDOWS

A SwiftUI app starts at a type marked `@main` that conforms to `App`. Its `body` returns one or more **scenes**: `WindowGroup` for the main window (multiple windows on iPad and Mac), `Settings` for a Mac preferences window, and others like `DocumentGroup` and `Window`. App-wide setup, such as SwiftData containers and environment values, attaches here.

NOTES

- One `@main` App type per target
- Xcode's App template creates `MyApp.swift` + `ContentView.swift`
- Scenes can be modified like views (`.modelContainer`, `.environment`)
- Use `@State` in the App for app-wide models

APP ENTRY POINT

```
import SwiftUI
import SwiftData

@main
struct NotesApp: App {
    @State private var store = NoteStore()      // app-wide model

    var body: some Scene {
        WindowGroup {
            ContentView()
                .environment(store)             // share with all views
        }
        .modelContainer(for: Note.self)         // SwiftData

        #if os(macOS)
        Settings { SettingsView() }             // ⌘, window on Mac
        #endif
    }
}
```

CODE EXPLAINED

- → `@main` + `App` — program entry
- → `some Scene` — scene builder, like `some View`
- → `WindowGroup` — main window(s)
- → `.environment(store)` — inject a model
- → `Settings` — macOS preferences scene

MINIMUM OS

- `App` / `Scene`: iOS 14 / macOS 11
- `Window` (single window), `openWindow`: macOS 13

UIKIT / REACT EQUIVALENT

- **`UIKit`** `AppDelegate` + `SceneDelegate` + root view controller
- **`React Native`** `App.tsx` root component registered with `AppRegistry`

SCENE TYPES

SCENE	USE
`WindowGroup`	Main UI, multiple windows
`Window`	One unique window (Mac)
`DocumentGroup`	Document-based apps
`Settings`	Mac preferences
`MenuBarExtra`	Mac menu bar app
`ImmersiveSpace`	visionOS spaces

⚠ COMMON MISTAKES

- Creating models inside `ContentView` with `@State` when the whole app needs them
- Two `@main` types in one target: compile error
- Using `Settings` on iOS: it's Mac-only

✓ TIPS & BEST PRACTICES

- Use `@UIApplicationDelegateAdaptor` for push notification setup (Sheet 39)
- Keep `App` small: create models, inject them, nothing else
- Add `.commands { }` to a scene for Mac menu items

QUICK SUMMARY

- `@main struct X: App` — entry
- `WindowGroup` — main scene
- inject app-wide state at the scene

Views & body

THE VIEW PROTOCOL

A view is a struct that conforms to `View` and provides a computed `body`. `body` returns `some View`: one concrete type the compiler works out from your code. Build screens by **composing** small views; extracting a subview into its own struct is cheap and makes previews and reuse easy. `body` is implicitly a `@ViewBuilder`, so you can list views and use `if` and `switch` inside it.

NOTES

- Views should be small; SwiftUI handles the cost
- Pass data into subviews through their properties (`let`)
- Use `@ViewBuilder` on your own functions that return views
- `Group` groups views without adding layout
- Swift set: O50 · Result Builders, O74 · Opaque & Existential Types

COMPOSING VIEWS

```
struct ProfileView: View {
    let user: User
    var body: some View {
        VStack(alignment: .leading) {
            Avatar(url: user.photoURL)              // subview
            Text(user.name).font(.title2)
            if user.isPro { ProBadge() }            // conditional
            stats                                   // computed sub-view
        }
    }
    @ViewBuilder private var stats: some View {
        if user.posts > 0 { Text("\(user.posts) posts") }
        else { Text("No posts yet").foregroundStyle(.secondary) }
    }
}

struct ProBadge: View {
    var body: some View { Text("PRO").font(.caption.bold()) }
}

#Preview { ProfileView(user: .sample) }
```

CODE EXPLAINED

- → `some View` — opaque return type
- → subview structs — reusable, previewable
- → `if` in body — conditional content
- → `@ViewBuilder` property — multi-branch helper
- → `let user` — data passed in

MINIMUM OS

- `View` , `@ViewBuilder` : iOS 13
- `if` / `switch` in builders: iOS 14+

BODY BUILDING BLOCKS

CONSTRUCT	PURPOSE
`Group { }`	Group views, no layout
`if / else`	Conditional views
`switch`	One view per case
`ForEach`	Views from a collection
`EmptyView()`	Nothing

UIKIT / REACT EQUIVALENT

- **`UIKit`** Custom `UIView` subclasses + `addSubview`
- **`React Native`** Small function components with props

⚠ COMMON MISTAKES

- One 300-line `body` : slow to type-check and hard to read
- Returning different types from a non- `@ViewBuilder` function: "opaque return type" error
- Using `AnyView` to mix types: hurts diffing; use `@ViewBuilder`

✓ TIPS & BEST PRACTICES

- Extract a subview when a block gets a comment heading
- Prefer subview structs over computed properties for reuse
- Give previews sample data with a `static let sample`

QUICK SUMMARY

- `struct X: View { var body }`
- compose small views
- `@ViewBuilder` for multi-branch helpers

Previews

LIVE PREVIEWS

The `#Preview` macro shows a view in Xcode's canvas, updating as you type. You can declare several previews per file, give them names, and apply **traits** such as landscape orientation or a fixed size. `@Previewable` lets a preview use `@State` directly, so you can preview views that need a binding.

NOTES

- Previews run your real code in a small app process
- Pin a preview to keep it visible while editing other files
- Use sample data, never live network calls
- Canvas modes: Live (interactive) and Selectable
- Swift set: O96 · Macros

#PREVIEW VARIATIONS

```
#Preview { CounterView() }

#Preview("Dark, large text") {
    CounterView()
        .preferredColorScheme(.dark)
        .dynamicTypeSize(.xxxLarge)
}

#Preview(traits: .landscapeLeft) { Dashboard() }
#Preview(traits: .sizeThatFitsLayout) { ProBadge() }

#Preview("Toggle row") {
    @Previewable @State var isOn = true       // state inside a preview
    SettingRow(title: "Wi-Fi", isOn: $isOn)
}

#Preview("With data") {
    NoteList()
        .modelContainer(for: Note.self, inMemory: true)
}
```

CODE EXPLAINED

- → `#Preview("name")` — named preview
- → `traits:` — orientation, size
- → `@Previewable @State` — bindable preview state
- → `inMemory: true` — throwaway SwiftData store
- → modifiers — dark mode, Dynamic Type

MINIMUM OS

- `#Preview` macro: Xcode 15
- `@Previewable` : Xcode 16

UIKIT / REACT EQUIVALENT

- **UIKit** `#Preview { MyViewController() }` works for UIKit too
- **React Native** Storybook stories ≈ named previews

PREVIEW TRAITS

TRAIT	EFFECT
`.portrait / .landscapeLeft`	Orientation
`.sizeThatFitsLayout`	Just the view
`.fixedLayout(width:height:)`	Exact size
`.modifier(SampleData())`	Shared setup (iOS 18)

⚠ COMMON MISTAKES

- Calling the network or real database in previews: slow and flaky
- Old `PreviewProvider` structs in new code: use `#Preview`
- A crashing preview with no idea why: check the diagnostics button

✓ TIPS & BEST PRACTICES

- Create `static let sample` / `.mock` values on models for previews
- Preview dark mode and large Dynamic Type every time
- ⌥⌘↩ toggles the canvas; ⌥⌘P refreshes it

QUICK SUMMARY

- `#Preview { View() }`
- traits + modifiers for variants
- `@Previewable` for state

Modifiers & Order

HOW MODIFIERS WORK

A modifier like `.padding()` doesn't change a view; it **wraps** it in a new view. That's why **order matters**: `.padding().background(.blue)` colors the padding, while `.background(.blue).padding()` doesn't. Read a modifier chain from top to bottom as layers added from the inside out.

NOTES

- Each modifier returns a new view type
- Some modifiers flow down to children (`.font` , `.foregroundStyle`)
- Layout modifiers (`padding` , `frame`) change size; others just draw
- Avoid `if` around modifiers: it changes view identity

ORDER MATTERS

```
Text("A")
    .padding()                      // 1. add space
    .background(.yellow)            // 2. color includes padding
    .border(.red)                   // 3. border around both

Text("B")
    .background(.yellow)            // 1. color just the text
    .padding()                      // 2. space outside the color
    .border(.red)

VStack {                            // environment flows down
    Text("Title")
    Text("Subtitle")
}
.font(.headline)                    // both Texts get .headline
.foregroundStyle(.blue)

// conditional value, not conditional modifier
Text("Status")
    .opacity(isHidden ? 0 : 1)      // same identity either way
```

CODE EXPLAINED

- → modifier = new wrapping view
- → `padding` then `background` — color fills padding
- → `.font` on a container — applies to children
- → ternary inside modifier — keeps identity stable

MINIMUM OS

- Core modifiers: iOS 13
- `foregroundStyle` replaces `foregroundColor` : iOS 15

UIKIT / REACT EQUIVALENT

- **`UIKit`** Setting properties on a view (`layer.borderColor` , `backgroundColor`)
- **`React Native`** `style={{ padding, backgroundColor }}` , but order-free

COMMON MODIFIERS

MODIFIER	DOES
`.padding(_:)`	Space around
`.frame(width:height:)`	Propose size
`.background(_:)`	Layer behind
`.overlay(_:)`	Layer in front
`.clipShape(.rect(cornerRadius: 8))`	Clip
`.shadow(radius:)`	Drop shadow

⚠ COMMON MISTAKES

- `.background(...)` before `.padding()` and wondering why the color is tight
- `if flag { view.bold() } else { view }` : two identities, broken animations
- Applying `.frame` after `.background` and expecting the background to grow

✓ TIPS & BEST PRACTICES

- When layout looks wrong, add `.border(.red)` between modifiers to see each layer
- Bundle repeated chains into a custom `ViewModifier` (Sheet 47)
- Put modifiers that apply to many children on the container

QUICK SUMMARY

- modifiers wrap views
- order changes the result
- some modifiers flow to children

Text & Labels

DISPLAYING TEXT

`Text` displays read-only strings, styled with modifiers like `.font`, `.fontWeight` and `.foregroundStyle`. String literals in `Text` are **localized** automatically and support simple **Markdown** (bold, italic, links). `Label` pairs text with an icon and adapts to its context, for example showing only the icon in a toolbar.

NOTES

- Use semantic fonts (`.headline` , `.body`) so Dynamic Type works
- `Text` concatenation with `+` mixes styles
- `Text(date, style: .relative)` updates live
- `AttributedString` for rich text from code

TEXT & LABEL

```
VStack(alignment: .leading, spacing: 8) {
    Text("Welcome back").font(.largeTitle.bold())
    Text("Last sync \(lastSync, style: .relative) ago")
        .font(.subheadline)
        .foregroundStyle(.secondary)
    Text("Read the **terms** and [privacy policy](https://ex.com)")
    Text(longText)
        .lineLimit(2)
        .truncationMode(.tail)
    Text(price, format: .currency(code: "USD"))
    Text("Total: ").bold() + Text("$42").foregroundStyle(.green)
    Label("Downloads", systemImage: "arrow.down.circle")
    Text(verbatim: "Not_localized_key")
}
```

CODE EXPLAINED

- → `.font(.largeTitle.bold())` — semantic font + weight
- → `style: .relative` — live-updating date
- → Markdown — bold, links in literals
- → `lineLimit` / `truncationMode` — overflow
- → `format:` — FormatStyle in Text

MINIMUM OS

- `Text` Markdown, `format:` : iOS 15
- `Label` : iOS 14

UIKIT / REACT EQUIVALENT

- **UIKit** `UILabel` + `NSAttributedString`
- **React Native** `<Text style={…}>` , nested `<Text>` for mixed styles

ACCESSIBILITY

- ♿ Semantic fonts scale with Dynamic Type automatically
- ♿ `Label` reads its title to VoiceOver even when only the icon shows

TEXT STYLES

FONT	USE
`.largeTitle`	Screen titles
`.title / .title2 / .title3`	Section titles
`.headline`	Emphasized rows
`.body`	Default text
`.caption`	Small notes

⚠ COMMON MISTAKES

- Fixed sizes like `.font(.system(size: 14))` : ignores Dynamic Type
- Markdown from a variable doesn't render: wrap it in `AttributedString(markdown:)`
- Concatenating strings for display when `Text(… format:)` would localize

✓ TIPS & BEST PRACTICES

- Use `.monospacedDigit()` for counters that shouldn't jiggle
- Use `Text(verbatim:)` for user content that shouldn't be localized
- `.textSelection(.enabled)` lets users copy text

QUICK SUMMARY

- `Text` + semantic fonts
- Markdown and `format:` built in
- `Label` = text + icon

Images & SF Symbols

IMAGES & SF SYMBOLS

`Image` shows images from the asset catalog or **SF Symbols**, Apple's library of thousands of icons that scale with text. Images draw at their natural size until you add `.resizable()` and a content mode. `AsyncImage` loads remote images with a placeholder. Symbols support rendering modes, variable values and animated **symbol effects**.

NOTES

- Browse symbols in the free SF Symbols app
- Symbols size with `.font`, not `.frame`
- `.resizable()` must come first among image modifiers
- Decorative images should be hidden from VoiceOver

IMAGES & SYMBOLS

```
Image("hero")                              // asset catalog
    .resizable()
    .scaledToFill()
    .frame(height: 200)
    .clipped()

Image(systemName: "heart.fill")
    .font(.title)                          // symbols scale with font
    .foregroundStyle(.red)

Image(systemName: "cloud.sun.rain.fill")
    .symbolRenderingMode(.multicolor)

Image(systemName: "bell")
    .symbolEffect(.bounce, value: unread)  // animate on change

AsyncImage(url: user.avatarURL) { image in
    image.resizable().scaledToFit()
} placeholder: { ProgressView() }
.frame(width: 64, height: 64)
.clipShape(.circle)
```

CODE EXPLAINED

- → `.resizable()` + `scaledToFill/Fit` — sizing
- → `systemName:` — SF Symbol
- → `symbolRenderingMode` — mono, hierarchical, multicolor
- → `symbolEffect` — animated symbols
- → `AsyncImage` — remote image + placeholder

MINIMUM OS

- SF Symbols: iOS 13; `AsyncImage`: iOS 15
- `symbolEffect`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UIImageView` + `UIImage(systemName:)`
- **React Native** `<Image source={{uri}} />` + an icon library

ACCESSIBILITY

- ♿ Hide decorative images: `.accessibilityHidden(true)`
- ♿ Give meaningful images a label: `Image("x", label: Text("…"))`

RENDERING MODES

MODE	LOOK
`.monochrome`	One color
`.hierarchical`	One color, layered opacity
`.palette`	Your own colors
`.multicolor`	Symbol's built-in colors

⚠ COMMON MISTAKES

- Forgetting `.resizable()`: the image ignores your frame
- Sizing symbols with `.frame`: blurry or misaligned; use `.font`
- Using `AsyncImage` for large scrolling lists: no caching beyond URLCache

✓ TIPS & BEST PRACTICES

- Use `.imageScale(.large)` to nudge symbol size inside text
- Use Nuke or Kingfisher when you need real image caching
- Export custom icons as custom SF Symbols to match text weight

QUICK SUMMARY

- `Image("asset")` / `Image(systemName:)`
- `.resizable()` + content mode
- `AsyncImage` for URLs

Colors, Gradients & Materials

COLOR, GRADIENTS & MATERIALS

`Color` values come from the system (`.blue`, `.primary`, `.secondary`), the asset catalog (with light and dark variants), or RGB. Apply them with `.foregroundStyle`, `.background` and `.tint`. Gradients and **materials** (translucent blurred backgrounds) are also `ShapeStyle`s, so they work anywhere a color does.

NOTES

- `.primary` / `.secondary` adapt to light and dark mode
- `.tint` sets the accent for controls in a subtree
- Asset catalog colors: `Color("Brand")` or generated `Color.brand`
- Materials blur what's behind them

STYLES

```
Text("Primary").foregroundStyle(.primary)
Text("Muted").foregroundStyle(.secondary)
Text("Brand").foregroundStyle(Color.brand)       // asset symbol

Rectangle()
    .fill(.linearGradient(colors: [.purple, .blue],
                          startPoint: .top, endPoint: .bottom))
    .frame(height: 120)

Circle().fill(.radialGradient(colors: [.yellow, .orange],
                              center: .center, startRadius: 5, endRadius: 60))

Text("Overlay")
    .padding()
    .background(.ultraThinMaterial, in: .rect(cornerRadius: 12))

Form { Toggle("Alerts", isOn: $on) }
    .tint(.green)                                // control accent

Color(red: 0.2, green: 0.6, blue: 0.9).opacity(0.5)
```

CODE EXPLAINED

- → `.primary` / `.secondary` — adaptive text colors
- → `Color.brand` — generated asset color symbol
- → `.linearGradient` / `.radialGradient` — gradient styles
- → `.ultraThinMaterial` — blurred background
- → `.tint` — accent color for controls

MINIMUM OS

- Materials, `foregroundStyle`: iOS 15
- Asset color symbols (`Color.brand`): Xcode 15

SEMANTIC COLORS

COLOR	USE
`.primary`	Main text
`.secondary`	Supporting text
`.accentColor`	App accent
`Color(.systemBackground)`	Screen background
`Color(.secondarySystemBackground)`	Grouped cards

UIKIT / REACT EQUIVALENT

- **UIKit** `UIColor.systemBlue`, `UIVisualEffectView` for blur
- **React Native** Hex strings in styles; `expo-blur` for materials

ACCESSIBILITY

- ♿ Check contrast in both appearances; asset colors support high-contrast variants
- ♿ Don't use color alone to convey state; add an icon or text

⚠ COMMON MISTAKES

- Hard-coding `.black` text: invisible in dark mode
- Using `foregroundColor`: deprecated, use `foregroundStyle`
- Stacking several materials: muddy and slow

✓ TIPS & BEST PRACTICES

- Define brand colors in the asset catalog with dark variants
- Use `.background(.regularMaterial, in: .capsule)` for floating chips
- `ShapeStyle` lets one property accept colors, gradients or materials

QUICK SUMMARY

- semantic + asset colors
- gradients and materials are styles
- `.tint` for controls

Stacks

STACKING VIEWS

Stacks are the core layout tools: `VStack` arranges children vertically, `HStack` horizontally and `ZStack` on top of each other. Each takes an `alignment` and (for V/H) a `spacing`. `Spacer` expands to push views apart, and `Divider` draws a separator line in the stack's direction.

NOTES

- Stacks size to fit their children
- `Spacer()` takes all free space; `Spacer(minLength:)` sets a floor
- Default spacing is adaptive per platform
- Use lazy stacks only inside scroll views (Sheet 28)

V, H & Z STACKS

```
HStack(alignment: .top, spacing: 12) {
    Image(systemName: "person.circle.fill")
        .font(.largeTitle)
    VStack(alignment: .leading, spacing: 4) {
        Text("Ada Lovelace").font(.headline)
        Text("Analyst").foregroundStyle(.secondary)
    }
    Spacer()                                    // push button right
    Button("Follow") { }
}

VStack {
    Text("Top")
    Divider()
    Text("Bottom")
}

ZStack(alignment: .bottomTrailing) {
    Image("photo").resizable().scaledToFit()
    Text("NEW").padding(6).background(.red)     // badge in corner
}
```

CODE EXPLAINED

- → `HStack(alignment:spacing:)` — row
- → `VStack(alignment: .leading)` — left-aligned column
- → `Spacer()` — fill remaining space
- → `Divider()` — separator line
- → `ZStack(alignment:)` — layered views

MINIMUM OS

- Stacks, `Spacer`, `Divider`: iOS 13
- `LazyVStack` / `LazyHStack`: iOS 14

UIKIT / REACT EQUIVALENT

- **UIKit** `UIStackView` (axis, alignment, spacing)
- **React Native** `<View>` with `flexDirection: 'row' | 'column'`

ALIGNMENT VALUES

STACK	OPTIONS
VStack	.leading .center .trailing
HStack	.top .center .bottom .firstTextBaseline
ZStack	.topLeading … .bottomTrailing

⚠ COMMON MISTAKES

- Expecting `VStack` to fill the width: it hugs its content; add `.frame(maxWidth: .infinity)`
- Using `ZStack` for side-by-side content
- Deeply nesting stacks where a `Grid` would be clearer

✓ TIPS & BEST PRACTICES

- Use `.firstTextBaseline` alignment for text of different sizes in a row
- Pass `spacing: 0` and add padding yourself for precise designs
- Prefer `.overlay` over `ZStack` when one view decorates another

QUICK SUMMARY

- `VStack` / `HStack` / `ZStack`
- `Spacer` pushes, `Divider` separates
- alignment + spacing parameters

Padding, Frames & Alignment

SIZING & POSITIONING

SwiftUI layout is a negotiation: a parent **proposes** a size, the child **chooses** its size, and the parent **places** it. `.padding` adds space, `.frame` proposes a size (fixed or min/max) and aligns the content within it, and `fixedSize` and `layoutPriority` settle conflicts when space runs out.

NOTES

- `.frame(maxWidth: .infinity)` makes a view as wide as allowed
- Text wraps by default; `fixedSize()` stops truncation
- Higher `layoutPriority` gets space first
- `alignmentGuide` shifts a view's alignment point

PADDING & FRAMES

```
Text("Edge to edge")
    .padding(.horizontal, 16)
    .padding(.vertical, 8)
    .frame(maxWidth: .infinity, alignment: .leading)   // full width
    .background(.blue.opacity(0.1))

Image(systemName: "star")
    .frame(width: 44, height: 44)                      // tap target

Text("Fits")
    .frame(minWidth: 80, maxWidth: 200, minHeight: 32)

HStack {
    Text("A very long title that should win").layoutPriority(1)
    Text("Detail")                                     // shrinks first
}

Text("Never truncate me").fixedSize()

HStack(alignment: .top) {
    Text("•")
    Text("Shifted").alignmentGuide(.top) { d in d[.top] - 4 }
}
```

CODE EXPLAINED

- → `.padding(.horizontal, 16)` — edges + amount
- → `maxWidth: .infinity` — expand
- → `frame(width:height:)` — fixed size
- → `layoutPriority(1)` — gets space first
- → `fixedSize()` — ideal size, no truncation

MINIMUM OS

- All APIs here: iOS 13
- `.padding(.vertical, 8)` edge sets: iOS 13

FRAME FORMS

FORM	EFFECT
`.frame(width: 100)`	Fixed width
`.frame(maxWidth: .infinity)`	Fill width
`.frame(minHeight: 44)`	At least 44
`.frame(…, alignment: .leading)`	Position inside
`.fixedSize(horizontal: false, vertical: true)`	Grow vertically

UIKIT / REACT EQUIVALENT

- **`UIKit`** Auto Layout constraints, content hugging / compression priority
- **`React Native`** `padding`, `width`, `flex: 1`, `alignSelf`

⚠ COMMON MISTAKES

- Fixed widths everywhere: breaks on small phones and large text
- `.frame` after `.background`: the background doesn't grow
- Fighting truncation with fixed widths instead of `layoutPriority`

✓ TIPS & BEST PRACTICES

- Keep tap targets at least 44×44 points
- Use `.containerRelativeFrame` for "half the screen" sizes (Sheet 13)
- Debug sizes with `.border(.red)`

QUICK SUMMARY

- parent proposes, child chooses
- `padding` / `frame` / `maxWidth: .infinity`
- `layoutPriority` / `fixedSize` for conflicts

Overlays, Backgrounds & Safe Area

LAYERS & THE SAFE AREA

`.background` and `.overlay` add views behind or in front of a view and **take its size**, which makes them better than `ZStack` for decorations like badges and cards. The **safe area** keeps content clear of the notch, home indicator and bars; `ignoresSafeArea` extends backgrounds into it, and `safeAreaInset` adds your own bars that content scrolls under.

NOTES

- Overlay content is sized and aligned to the base view
- Backgrounds that are Colors fill the safe area automatically
- `safeAreaInset` shrinks the safe area for siblings like scroll views
- `zIndex` changes stacking order in a ZStack

OVERLAY, BACKGROUND, SAFE AREA

```
Image(systemName: "bell")
    .font(.title)
    .overlay(alignment: .topTrailing) {
        Circle().fill(.red).frame(width: 10, height: 10)    // badge
    }

Text("Card")
    .padding()
    .frame(maxWidth: .infinity)
    .background(.background, in: .rect(cornerRadius: 16))
    .shadow(radius: 4)

ZStack {
    Color.indigo.ignoresSafeArea()             // full-bleed background
    Text("Content stays safe")
}

ScrollView { messages }
    .safeAreaInset(edge: .bottom) {
        ComposeBar().background(.bar)          // list scrolls under it
    }
    .safeAreaPadding(.horizontal, 20)
```

CODE EXPLAINED

- → `.overlay(alignment:)` — decorate on top
- → `.background(_, in: shape)` — shaped background
- → `.ignoresSafeArea()` — extend into edges
- → `.safeAreaInset(edge:)` — custom bar
- → `.safeAreaPadding` — padding that respects scroll

MINIMUM OS

- `overlay(alignment:content:)` : iOS 15
- `safeAreaPadding` : iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** Subviews + `safeAreaLayoutGuide`, `additionalSafeAreaInsets`
- **React Native** `position: 'absolute'` + `SafeAreaView` / safe-area-context

CHOOSING A LAYER TOOL

TOOL	SIZE COMES FROM
`.background { }`	The base view
`.overlay { }`	The base view
`ZStack`	Largest child
`safeAreaInset`	Its own content

⚠ COMMON MISTAKES

- Calling `ignoresSafeArea` on content, not just its background: text under the notch
- Using ZStack for a badge and fighting alignment
- Adding a bottom bar with VStack over a ScrollView: content can't scroll under it

✓ TIPS & BEST PRACTICES

- `.ignoresSafeArea(.keyboard)` stops the keyboard pushing a view up
- Use `.background(.bar)` for system-styled bars
- Use `.containerShape` + `ContainerRelativeShape` for nested corner radii

QUICK SUMMARY

- `.overlay` / `.background` take the base size
- `ignoresSafeArea` for backgrounds
- `safeAreaInset` for custom bars

Grids

GRID LAYOUTS

`Grid` lays out a **fixed** set of rows and columns that align across rows, like a table. For **large or dynamic** collections, `LazyVGrid` and `LazyHGrid` create cells only as they scroll into view, with columns described by `GridItem`: fixed, flexible, or **adaptive** (fit as many as possible).

NOTES

- `Grid` is eager: fine for forms and dashboards
- Lazy grids must sit inside a `ScrollView`
- `.adaptive(minimum:)` gives a responsive photo grid
- `gridCellColumns(_:)` spans a cell across columns

GRID & LAZYVGRID

```
Grid(alignment: .leading, horizontalSpacing: 16) {
    GridRow {
        Text("Steps");  Text("8,204").bold()
    }
    GridRow {
        Text("Distance");  Text("6.1 km").bold()
    }
    Divider().gridCellColumns(2)
}

let columns = [GridItem(.adaptive(minimum: 100), spacing: 8)]

ScrollView {
    LazyVGrid(columns: columns, spacing: 8) {
        ForEach(photos) { photo in
            PhotoThumb(photo)
                .aspectRatio(1, contentMode: .fill)
        }
    }
    .padding(8)
}
```

CODE EXPLAINED

- → `Grid` + `GridRow` — aligned table layout
- → `gridCellColumns(2)` — span columns
- → `GridItem(.adaptive(minimum:))` — responsive columns
- → `LazyVGrid` in `ScrollView` — lazy cells

MINIMUM OS

- `LazyVGrid` / `LazyHGrid`: iOS 14
- `Grid` / `GridRow`: iOS 16

UIKIT / REACT EQUIVALENT

- **UIKit** `UICollectionView` with a compositional layout
- **React Native** `FlatList numColumns={3}`

GRIDITEM SIZES

SIZE	BEHAVIOR
`.fixed(80)`	Exactly 80 pt
`.flexible()`	Share space evenly
`.flexible(minimum: 50)`	Share, with a floor
`.adaptive(minimum: 100)`	As many as fit

⚠ COMMON MISTAKES

- Using `Grid` for hundreds of items: everything is built at once
- Lazy grid outside a ScrollView: content is clipped
- Expecting `.adaptive` columns to be one `GridItem` per column: one item makes many columns

✓ TIPS & BEST PRACTICES

- Use `GridRow(alignment:)` and `.gridColumnAlignment` for per-column alignment
- Add `pinnedViews: [.sectionHeaders]` for sticky headers
- Give cells a fixed aspect ratio to avoid jumpy scrolling

QUICK SUMMARY

- `Grid` — small aligned tables
- `LazyVGrid` — big scrolling collections
- `GridItem` — fixed / flexible / adaptive

Adaptive Layout

LAYOUTS THAT ADAPT

Screens range from Apple Watch to a Mac window, so layouts must adapt. `ViewThatFits` picks the first child that fits. `horizontalSizeClass` tells you compact vs regular width. `containerRelativeFrame` sizes a view as a fraction of its container. `GeometryReader` exposes exact sizes when nothing else works, and the `Layout` protocol builds custom layouts; `AnyLayout` switches between them with animation.

NOTES

- Prefer size classes and `ViewThatFits` over raw geometry
- `GeometryReader` takes all offered space: wrap it carefully
- `AnyLayout` keeps identity when switching H ↔ V stacks
- Custom `Layout` needs `sizeThatFits` + `placeSubviews`

ADAPTIVE TECHNIQUES

```
@Environment(\.horizontalSizeClass) private var sizeClass

var body: some View {
    let layout = sizeClass == .compact
        ? AnyLayout(VStackLayout()) : AnyLayout(HStackLayout())
    layout { ChartView(); DetailsView() }     // animates on change
}

ViewThatFits {                                // first that fits wins
    HStack { Label("Download", systemImage: "arrow.down"); Text("2.1 GB") }
    Label("Download", systemImage: "arrow.down")
    Image(systemName: "arrow.down")
}

ScrollView(.horizontal) {
    LazyHStack {
        ForEach(cards) { CardView($0) }
            .containerRelativeFrame(.horizontal, count: 3, spacing: 10)
    }
}

GeometryReader { geo in
    Ring(progress: p).frame(width: geo.size.width * 0.5)
}
```

CODE EXPLAINED

- → `horizontalSizeClass` — compact vs regular
- → `AnyLayout` — switch layouts, keep identity
- → `ViewThatFits` — fallbacks by size
- → `containerRelativeFrame(count:)` — 3 per screen
- → `GeometryReader` — exact size as a last resort

MINIMUM OS

- `ViewThatFits`, `AnyLayout`, `Layout`: iOS 16
- `containerRelativeFrame`: iOS 17

WHICH TOOL

NEED	USE
Phone vs iPad layout	horizontalSizeClass
Shorter label if tight	ViewThatFits
N items per screen width	containerRelativeFrame
Exact pixel math	GeometryReader
Flow / radial layout	Layout protocol

UIKIT / REACT EQUIVALENT

- **UIKit** Trait collections + `viewWillTransition(to:with:)`
- **React Native** `useWindowDimensions()` + conditional styles

⚠ COMMON MISTAKES

- Wrapping whole screens in `GeometryReader`: it breaks normal sizing
- Checking `UIScreen.main.bounds`: wrong in split view and on iPad windows
- Using `if compact { VStack } else { HStack }`: loses state; use `AnyLayout`

✓ TIPS & BEST PRACTICES

- Test in iPad split view and Slide Over, not just full screen
- Use `onGeometryChange` (iOS 18) to read a size without GeometryReader
- Write a small `FlowLayout` with the `Layout` protocol for tag clouds

QUICK SUMMARY

- size classes + `ViewThatFits`
- `containerRelativeFrame` for fractions
- `GeometryReader` / `Layout` when needed

@State

LOCAL VIEW STATE

`@State` stores a value **owned by a view**. SwiftUI keeps the storage outside the struct, so it survives when `body` is recomputed, and changing it schedules a redraw. Mark it `private`: it belongs to this view alone. State lives as long as the view's **identity**; if the view is removed or its identity changes, the state resets.

NOTES

- Use for simple, local values: flags, text input, selection
- Initialize with a default; don't set it from `init` parameters
- `.id(x)` changes identity and resets state on purpose
- Holds `@Observable` class instances too (Sheet 16)
- Swift set: O95 · Property Wrappers

@STATE

```
struct SearchBox: View {
    @State private var query = ""
    @State private var isExpanded = false
    @State private var taps = 0

    var body: some View {
        VStack(alignment: .leading) {
            TextField("Search", text: $query)       // binding to state
            Button(isExpanded ? "Less" : "More") {
                isExpanded.toggle()                  // triggers redraw
                taps += 1
            }
            if isExpanded { Text("Typed: \(query)") }
            Text("Tapped \(taps) times").font(.caption)
        }
        .padding()
    }
}

#Preview { SearchBox() }
```

CODE EXPLAINED

→ `@State private var` — view-owned value

→ `$query` — binding for child controls

→ mutation in an action — schedules a redraw

→ conditional UI from state

MINIMUM OS

- `@State`: iOS 13
- `@State` holding an `@Observable` class: iOS 17

STATE LIFETIME

EVENT	STATE
`body` recomputed	Kept
Parent re-renders	Kept (same identity)
View removed by `if`	Discarded
`.id(newValue)`	Reset
Different `ForEach` id	New state

UIKIT / REACT EQUIVALENT

- **UIKit** Stored properties + manual `label.text = …` updates
- **React Native** `const [query, setQuery] = useState("")`

⚠ COMMON MISTAKES

- Initializing `@State` from an init parameter: later parent changes are ignored
- Using `@State` for data other views own: pass a `@Binding` or model instead
- Making `@State` non-private and setting it from outside

✓ TIPS & BEST PRACTICES

- Keep state as close as possible to where it's used
- Use `.id(item.id)` to reset a form when switching items
- Group related state into a small struct

QUICK SUMMARY

- `@State` — view-owned storage
- changes trigger redraws
- lives as long as view identity

@Binding

TWO-WAY CONNECTIONS

A `@Binding` gives a child view **read and write** access to state owned elsewhere, without owning it. The parent passes `$value` (the projected binding of its `@State`), and when the child changes it, the parent's state changes. Controls like `Toggle` and `TextField` all take bindings.

NOTES

- Child declares `@Binding var x`; caller passes `$x`
- `.constant(value)` makes a read-only binding for previews
- Bindings to properties: `$settings.isOn`
- Custom `Binding(get:set:)` adapts or validates values
- Swift set: O95 · Property Wrappers

@BINDING

```
struct SettingsView: View {
    @State private var wifiOn = true
    @State private var volume = 0.5

    var body: some View {
        Form {
            SettingRow(title: "Wi-Fi", isOn: $wifiOn)   // pass binding
            VolumeSlider(value: $volume)
            Text(wifiOn ? "Connected" : "Off")
        }
    }
}

struct SettingRow: View {
    let title: String
    @Binding var isOn: Bool                              // doesn't own it
    var body: some View { Toggle(title, isOn: $isOn) }
}

let percent = Binding(                                   // adapter
    get: { Int(volume * 100) },
    set: { volume = Double($0) / 100 })

#Preview { SettingRow(title: "Demo", isOn: .constant(true)) }
```

CODE EXPLAINED

- → `@Binding var isOn` — borrowed read/write value
- → `$wifiOn` — projected binding from `@State`
- → `Toggle(isOn: $isOn)` — controls take bindings
- → `Binding(get:set:)` — computed binding
- → `.constant(true)` — fixed binding for previews

MINIMUM OS

- `@Binding`: iOS 13
- `ForEach($items) { $item in }` bindings: Xcode 13+

WHERE BINDINGS COME FROM

SOURCE	BINDING
`@State var x`	$x
`@Bindable var model`	$model.name
`@Binding var x`	$x (pass it on)
`ForEach($items) { $item in }`	$item
`Binding(get:set:)`	custom

UIKIT / REACT EQUIVALENT

- **`UIKit`** Delegate or closure callbacks to report changes back
- **`React Native`** Passing `value` + `onChange` props (lifted state)

⚠ COMMON MISTAKES

- Declaring `@State` in the child instead of `@Binding`: changes don't reach the parent
- Passing `wifiOn` instead of `$wifiOn`: type mismatch
- Using `.constant` outside previews: controls appear frozen

✓ TIPS & BEST PRACTICES

- Use `ForEach($items) { $item in TextField("", text: $item.name) }` to edit lists
- Prefer `@Bindable` models over many separate bindings
- Bindings are cheap; pass exactly the piece a child needs

QUICK SUMMARY

- `@Binding` — borrowed, writable
- `$x` creates one from state
- `Binding(get:set:)` for adapters

@Observable Models

OBSERVABLE MODEL CLASSES

For data shared by several views or with logic attached, use a class marked `@Observable`. SwiftUI tracks **which properties each view reads** in `body` and redraws only those views when those properties change. The view that creates the model holds it in `@State`; other views simply receive it as a normal property.

NOTES

- No `@Published` needed: every stored property is tracked
- Only properties read in `body` cause redraws
- `@ObservationIgnored` excludes a property from tracking
- Mark UI models `@MainActor`
- Swift set: O96 · Macros, O61 · Classes

@OBSERVABLE

```
import Observation

@MainActor @Observable
final class CartModel {
    var items: [Item] = []
    var couponCode = ""
    @ObservationIgnored var analytics = Analytics()   // not tracked

    var total: Decimal { items.reduce(0) { $0 + $1.price } }
    func add(_ item: Item) { items.append(item) }
}

struct ShopView: View {
    @State private var cart = CartModel()        // owner
    var body: some View {
        VStack {
            CartBadge(cart: cart)                // plain property
            Button("Add") { cart.add(.sample) }
        }
    }
}

struct CartBadge: View {
    let cart: CartModel                          // reads → tracked
    var body: some View { Text("\(cart.items.count) items") }
}
```

CODE EXPLAINED

→ `@Observable` — macro adds change tracking

→ `@State private var cart` — owning view keeps it alive

→ `let cart` — other views just read it

→ computed `total` — tracked through `items`

→ `@ObservationIgnored` — opt out

MINIMUM OS

- `@Observable` : iOS 17 / macOS 14
- Before iOS 17: `ObservableObject` (Sheet 19)

UIKIT / REACT EQUIVALENT

- **`UIKit`** Delegates, KVO, or Combine publishers to push updates
- **`React Native`** Zustand / MobX store; selectors limit re-renders

WHO HOLDS THE MODEL

VIEW ROLE	DECLARATION
Creates it	@State private var m = M()
Reads it	let m: M
Edits with bindings	@Bindable var m: M
Many views need it	@Environment(M.self)

⚠ COMMON MISTAKES

- Creating the model as a plain `let` in the owner: recreated on every parent update
- Mixing `@Published` into an `@Observable` class
- Mutating the model from a background thread without `@MainActor`

✓ TIPS & BEST PRACTICES

- Keep side effects (networking) in model methods, not views
- Store large immutable data in `let` to skip tracking overhead
- Use `withObservationTracking` outside SwiftUI if needed

QUICK SUMMARY

- `@Observable class` — shared model
- per-property redraws
- owner uses `@State`, others `let`

@Bindable & Sharing Models

BINDINGS INTO MODELS

Controls need bindings, but a model passed as `let` can't produce them. Declare it `@Bindable` in the view that edits it, then use `$model.property`. Ownership stays clear: one view **owns** the model in `@State`, editing views mark it `@Bindable`, and read-only views take it as `let`.

NOTES

- `@Bindable` works on any `@Observable` reference
- Create a local one in body: `@Bindable var m = model`
- Environment models need `@Bindable` locally to bind
- No copying: all views share one instance
- Swift set: O94 · Key Paths

@BINDABLE

```
@Observable final class Profile {
    var name = ""
    var bio = ""
    var notifications = true
}

struct EditProfileView: View {
    @Bindable var profile: Profile                  // can make bindings
    var body: some View {
        Form {
            TextField("Name", text: $profile.name)
            TextField("Bio", text: $profile.bio, axis: .vertical)
            Toggle("Notifications", isOn: $profile.notifications)
        }
    }
}

struct SettingsScreen: View {
    @Environment(Profile.self) private var profile
    var body: some View {
        @Bindable var profile = profile             // local bindable
        Toggle("Alerts", isOn: $profile.notifications)
    }
}

#Preview { EditProfileView(profile: Profile()) }
```

CODE EXPLAINED

→ `@Bindable var profile` — enables `$profile.x`

→ `$profile.name` — binding to a property

→ `@Environment(Profile.self)` — injected model

→ local `@Bindable var` — bind to an environment model

MINIMUM OS

- `@Bindable` : iOS 17 / macOS 14
- `TextField(axis: .vertical)` : iOS 16

UIKIT / REACT EQUIVALENT

- **`UIKit`** Target-action / delegates writing back to the model
- **`React Native`** Store actions like `setName(text)` from inputs

OWNERSHIP CHEAT SHEET

ROLE	WRAPPER
Creates and owns	@State
Displays only	let
Edits with controls	@Bindable
Reads from environment	@Environment(T.self)

⚠ COMMON MISTAKES

- Using `@State` in the editing view: it becomes a second owner
- Copying model fields into local `@State` and syncing them by hand
- Expecting `$model.x` to compile on a plain `let`

✓ TIPS & BEST PRACTICES

- For cancellable edits, edit a copy (struct draft) and save on Done
- Pass the narrowest thing: a binding for one field, the model for many
- Keep validation in the model so every view gets it

QUICK SUMMARY

- `@Bindable` → `$model.prop`
- owner `@State`, editors `@Bindable`
- local `@Bindable` for env models

Environment

THE ENVIRONMENT

The environment passes values **down the whole view tree** without threading them through every initializer. SwiftUI provides many built-in values (color scheme, locale, `dismiss`, `openURL`), and you can inject your own `@Observable` models with `.environment(model)` or define new keys with the `@Entry` macro.

NOTES

- Read with `@Environment(\.key)` or `@Environment(Type.self)`
- Values set on a view apply to all its descendants
- Missing environment model = runtime crash; inject it at the root
- Some values are actions: `dismiss()`, `openURL(url)`
- Swift set: 094 · Key Paths

READING & INJECTING

```
extension EnvironmentValues {
    @Entry var accentTheme: Theme = .ocean           // custom key
}

struct DetailView: View {
    @Environment(\.dismiss) private var dismiss
    @Environment(\.colorScheme) private var scheme
    @Environment(\.openURL) private var openURL
    @Environment(CartModel.self) private var cart   // injected model
    @Environment(\.accentTheme) private var theme

    var body: some View {
        VStack {
            Text(scheme == .dark ? "Dark" : "Light")
            Text("\(cart.items.count) in cart")
            Button("Help") { openURL(URL(string: "https://ex.com")!) }
            Button("Close") { dismiss() }
        }
        .tint(theme.color)
    }
}

// Injection somewhere above:
RootView().environment(cart).environment(\.accentTheme, .forest)
```

CODE EXPLAINED

- → `@Entry` — declare a custom environment value
- → `\.dismiss` — close the current presentation
- → `@Environment(CartModel.self)` — injected model
- → `.environment(cart)` — inject an Observable
- → `.environment(\.key, value)` — set a value

MINIMUM OS

- `@Environment(Type.self)` for Observables: iOS 17
- `@Entry` macro: Xcode 16

BUILT-IN VALUES

KEY	GIVES
`\.dismiss`	Close sheet / pop
`\.colorScheme`	.light / .dark
`\.horizontalSizeClass`	Compact / regular
`\.locale / \.calendar`	User settings
`\.scenePhase`	Active / background
`\.modelContext`	SwiftData context

UIKIT / REACT EQUIVALENT

- **`UIKit`** Trait collections, singletons, or passing dependencies by hand
- **`React Native`** `React.createContext` + `useContext`

⚠ COMMON MISTAKES

- Forgetting to inject an environment model: crash with "No Observable object of type…"
- Using the environment for data only one child needs: pass a property
- Calling `dismiss` from the wrong view (it closes that view's presentation)

✓ TIPS & BEST PRACTICES

- Inject app-wide models once in the `App` scene
- Previews need the same `.environment(...)` calls
- Use `@Entry` instead of writing EnvironmentKey boilerplate

QUICK SUMMARY

- `@Environment` reads down the tree
- `.environment(model)` injects
- `@Entry` for custom keys

ObservableObject (Legacy)

THE PRE-IOS 17 MODEL API

Before Observation, models conformed to `ObservableObject` and marked properties `@Published`. Views held them with `@StateObject` (owner), `@ObservedObject` (passed in) and `@EnvironmentObject` (injected). You'll meet this in older code and in apps supporting iOS 16. Unlike `@Observable`, **any** published change redraws **every** observing view.

NOTES

- Built on Combine's `objectWillChange` publisher
- `@StateObject` keeps the object alive across redraws
- `@ObservedObject` doesn't own: recreated if the parent recreates it
- Migration to `@Observable` is mostly mechanical

OBSERVABLEOBJECT

```swift
import Combine

final class TimerModel: ObservableObject {
    @Published var seconds = 0
    @Published var isRunning = false
}

struct TimerScreen: View {
    @StateObject private var timer = TimerModel()      // owner
    var body: some View {
        TimerFace(timer: timer)
            .environmentObject(timer)                  // inject
    }
}

struct TimerFace: View {
    @ObservedObject var timer: TimerModel              // passed in
    var body: some View { Text("\(timer.seconds)s") }
}

struct StopButton: View {
    @EnvironmentObject var timer: TimerModel           // injected
    var body: some View { Button("Stop") { timer.isRunning = false } }
}
```

CODE EXPLAINED

- → `ObservableObject` + `@Published` — old model
- → `@StateObject` — owner
- → `@ObservedObject` — non-owning reference
- → `.environmentObject` / `@EnvironmentObject` — injection

MINIMUM OS

- `ObservableObject`, `@ObservedObject`: iOS 13
- `@StateObject`: iOS 14

MIGRATION MAP

OLD	NEW
`ObservableObject + @Published`	`@Observable`
`@StateObject`	`@State`
`@ObservedObject`	`let / @Bindable`
`@EnvironmentObject`	`@Environment(T.self)`
`.environmentObject(x)`	`.environment(x)`

UIKIT / REACT EQUIVALENT

- **`UIKit`** Combine `sink` on `@Published` in view controllers
- **`React Native`** Redux store + `connect` / `useSelector`

⚠ COMMON MISTAKES

- Using `@ObservedObject` to create an object: it gets recreated and loses state
- Forgetting `@Published`: views don't update
- Mixing old and new wrappers on one object

✓ TIPS & BEST PRACTICES

- Migrate one model at a time; both systems coexist
- Use `@Published` + Combine only if you need publisher pipelines
- Check deployment target: below iOS 17, stay with this API

QUICK SUMMARY

- `ObservableObject` / `@Published` — legacy
- `@StateObject` owns, `@ObservedObject` borrows
- migrate to `@Observable`

Buttons

BUTTONS

`Button` runs an action when tapped. Give it a title and a trailing closure, or a custom label view. **Roles** (`.destructive`, `.cancel`) tell the system how to style and place it, and **button styles** (`.bordered`, `.borderedProminent`, `.plain`) set its look for a whole subtree.

NOTES

- Style a group at once: `.buttonStyle(.bordered)` on a container
- `.controlSize(.large)` scales built-in styles
- `.disabled(cond)` dims and blocks taps
- Async work: start a `Task` inside the action
- Swift set: O45 · Closures — Basics

BUTTON FORMS

```
Button("Save") { save() }
    .buttonStyle(.borderedProminent)
    .controlSize(.large)

Button("Delete", role: .destructive) { delete() }
    .buttonStyle(.bordered)

Button {
    Task { await refresh() }                    // async action
} label: {
    Label("Refresh", systemImage: "arrow.clockwise")
}
.disabled(isLoading)

Button(action: share) {
    Image(systemName: "square.and.arrow.up")
}
.accessibilityLabel("Share")

HStack { Button("A") {}; Button("B") {} }
    .buttonStyle(.plain)                        // applies to both
    .sensoryFeedback(.success, trigger: savedCount)
```

CODE EXPLAINED

- → `Button("Title") { }` — simplest form
- → `role: .destructive` — red, placed correctly
- → `label:` — any view as the label
- → `.disabled` — block taps
- → `.sensoryFeedback` — haptics on change

MINIMUM OS

- `role:`, `.bordered` styles: iOS 15
- `.sensoryFeedback`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UIButton` with `UIButton.Configuration` + `addAction`
- **React Native** `<Pressable onPress>` / `<Button title onPress>`

ACCESSIBILITY

- ♿ Icon-only buttons need `.accessibilityLabel`
- ♿ Keep the tap area at least 44×44 pt; add `.contentShape(.rect)` for padded labels

BUILT-IN STYLES

STYLE	LOOK
`.automatic`	Platform default
`.bordered`	Tinted capsule
`.borderedProminent`	Filled accent
`.borderless`	Text only
`.plain`	No styling at all

⚠ COMMON MISTAKES

- Wrapping a view in `.onTapGesture` instead of `Button`: loses accessibility and highlight
- Long-running work directly in the action without `Task`
- Tapping only the text of a wide button: add `.contentShape(.rect)`

✓ TIPS & BEST PRACTICES

- Write custom `ButtonStyle`s for brand buttons (Sheet 47)
- Use `.keyboardShortcut(.defaultAction)` on Mac/iPad
- Show progress by swapping the label while `isLoading`

QUICK SUMMARY

- `Button(title) { action }`
- roles + styles
- `.disabled` / `.controlSize`

Toggle, Slider & Stepper

SWITCHES & SLIDERS

`Toggle` binds to a `Bool`, `Slider` to a floating-point value within a range, and `Stepper` increments or decrements a value. All take **bindings** and adapt their look to the platform. Use `onEditingChanged` to know when a drag begins and ends, for example to save only when the user lets go.

NOTES

- Labels are required for accessibility, even if hidden
- `.labelsHidden()` hides the visual label but keeps it for VoiceOver
- `Slider(step:)` snaps to increments
- `toggleStyle(.button)` makes a toggle look like a button

TOGGLE, SLIDER, STEPPER

```
@State private var wifi = true
@State private var brightness = 0.7
@State private var guests = 2

Form {
    Toggle("Wi-Fi", isOn: $wifi)

    Slider(value: $brightness, in: 0...1) {
        Text("Brightness")
    } minimumValueLabel: {
        Image(systemName: "sun.min")
    } maximumValueLabel: {
        Image(systemName: "sun.max")
    } onEditingChanged: { editing in
        if !editing { save(brightness) }       // on release
    }

    Stepper("Guests: \(guests)", value: $guests, in: 1...10)

    Toggle("Bold", systemImage: "bold", isOn: $isBold)
        .toggleStyle(.button)
}
```

CODE EXPLAINED

- → `Toggle(isOn:)` — Bool binding
- → `Slider(value:in:)` — range binding
- → min/max labels — icons at each end
- → `onEditingChanged` — drag start/end
- → `Stepper(value:in:)` — +/- control

MINIMUM OS

- Toggle, Slider, Stepper: iOS 13
- `Toggle(_:systemImage:isOn:)`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UISwitch`, `UISlider`, `UIStepper` + value-changed actions
- **React Native** `<Switch>`, community `<Slider>`, custom stepper

ACCESSIBILITY

- ♿ VoiceOver reads the label and value: always give one, then `.labelsHidden()` if needed
- ♿ Sliders adjust with swipe up/down; give a `step` that makes sense

TOGGLE STYLES

STYLE	LOOK
`.automatic`	Switch (iOS)
`.switch`	Switch
`.button`	Highlighted button
`.checkbox`	Checkbox (macOS)

⚠ COMMON MISTAKES

- Saving on every slider tick: use `onEditingChanged` or debounce
- Using an empty label `Toggle("")`: VoiceOver says nothing
- Ranges on Int with `Slider`: it needs a floating-point binding

✓ TIPS & BEST PRACTICES

- Show the live value next to a slider with `.monospacedDigit()`
- Use `Gauge` for read-only progress values
- Wrap controls in `Form` for platform-standard layout

QUICK SUMMARY

- `Toggle` / `Slider` / `Stepper`
- all bind to state
- `onEditingChanged` for drag end

Pickers

CHOOSING FROM OPTIONS

`Picker` selects one value from a list; each option is tagged with the value it represents. Its look comes from `pickerStyle`: menu, segmented, wheel, inline or navigation link. `DatePicker` picks dates and times within a range, and `ColorPicker` opens the system color panel.

NOTES

- Selection type must match the `.tag` types exactly
- `ForEach` over an enum with `CaseIterable` fills a picker
- Optional selections need optional tags: `.tag(Optional(x))`
- Inside `Form`, the default style adapts to platform
- Swift set: 063 · Enums — Basics

PICKER, DATEPICKER, COLORPICKER

```
enum Size: String, CaseIterable, Identifiable {
    case small, medium, large
    var id: Self { self }
}
@State private var size = Size.medium
@State private var due = Date.now
@State private var color = Color.blue

Form {
    Picker("Size", selection: $size) {
        ForEach(Size.allCases) { s in
            Text(s.rawValue.capitalized).tag(s)
        }
    }
    .pickerStyle(.segmented)

    DatePicker("Due", selection: $due,
               in: Date.now..., displayedComponents: [.date, .hourAndMinute])

    ColorPicker("Accent", selection: $color, supportsOpacity: false)
}
```

CODE EXPLAINED

- → `Picker(selection:)` + `.tag` — options
- → `CaseIterable` enum — easy option list
- → `.pickerStyle(.segmented)` — style
- → `DatePicker(in:)` — limit the range
- → `displayedComponents` — date, time or both

MINIMUM OS

- `Picker`, `DatePicker`: iOS 13; `ColorPicker`: iOS 14
- `.pickerStyle(.palette)`: iOS 17

PICKER STYLES

STYLE	BEST FOR
`.menu`	Compact dropdown
`.segmented`	2–5 short options
`.wheel`	Spinning wheel
`.inline`	List of rows in a Form
`.navigationLink`	Push to a list

UIKIT / REACT EQUIVALENT

- **UIKit** `UISegmentedControl`, `UIPickerView`, `UIDatePicker`, `UIColorWell`
- **React Native** `@react-native-picker/picker`, datetimepicker libraries

ACCESSIBILITY

- Each option's `Text` is what VoiceOver reads; use full words
- Segmented pickers announce "1 of 3" automatically

⚠ COMMON MISTAKES

- Tag type differs from selection type (e.g. `String` vs enum): selection never changes
- Segmented picker with long labels: truncated
- DatePicker without `in:` when past dates aren't allowed

✓ TIPS & BEST PRACTICES

- Use `.datePickerStyle(.graphical)` for a calendar
- Use `MultiDatePicker` for picking several dates (iOS 16)
- `.labelsHidden()` for pickers that sit in custom rows

QUICK SUMMARY

- `Picker` + `.tag(value)`
- styles: menu / segmented / wheel
- `DatePicker` / `ColorPicker`

Text Input & Focus

TYPING & KEYBOARD FOCUS

`TextField` edits a String (or a formatted value), `SecureField` hides passwords, and `TextEditor` handles multi-line text. `@FocusState` tracks and moves keyboard focus between fields, and `onSubmit` reacts to the return key. Keyboard type and content type help the system offer the right keyboard and AutoFill.

NOTES

- `TextField(axis: .vertical)` grows as the user types
- `format:` lets a TextField edit numbers directly
- `.textContentType(.emailAddress)` enables AutoFill
- `.scrollDismissesKeyboard(.interactively)` on scroll views

FIELDS & FOCUS

```
enum Field { case email, password }
@State private var email = ""
@State private var password = ""
@State private var age = 0
@FocusState private var focus: Field?

Form {
    TextField("Email", text: $email)
        .keyboardType(.emailAddress)
        .textContentType(.emailAddress)
        .textInputAutocapitalization(.never)
        .focused($focus, equals: .email)
        .onSubmit { focus = .password }          // return → next
    SecureField("Password", text: $password)
        .focused($focus, equals: .password)
        .submitLabel(.go)
        .onSubmit(signIn)
    TextField("Age", value: $age, format: .number)
    TextField("Notes", text: $notes, axis: .vertical).lineLimit(3...6)
}
.onAppear { focus = .email }
```

CODE EXPLAINED

- → `@FocusState` — which field has focus
- → `.focused($focus, equals:)` — link a field
- → `.onSubmit` — return key action
- → `value:format:` — numeric field
- → `axis: .vertical` + `lineLimit(3...6)` — growing field

MINIMUM OS

- `@FocusState`, `onSubmit`: iOS 15
- `TextField(axis:)`: iOS 16

UIKIT / REACT EQUIVALENT

- **UIKit** `UITextField` delegates, `becomeFirstResponder()`
- **React Native** `<TextInput>` with refs, `ref.focus()`, `onSubmitEditing`

ACCESSIBILITY

- ♿ The placeholder is not a label: add `Text` labels or `LabeledContent` in custom layouts
- ♿ Use `textContentType` so password managers and VoiceOver identify fields

KEYBOARD MODIFIERS

MODIFIER	EFFECT
`.keyboardType(.decimalPad)`	Keyboard layout
`.textContentType(.oneTimeCode)`	AutoFill hint
`.submitLabel(.next)`	Return key text
`.autocorrectionDisabled()`	No autocorrect
`.textInputAutocapitalization(.never)`	No auto-caps

⚠ COMMON MISTAKES

- Using placeholder text as the only label
- Forgetting `.textInputAutocapitalization(.never)` on emails and usernames
- Dismissing the keyboard with UIKit hacks instead of `focus = nil`

✓ TIPS & BEST PRACTICES

- Set `focus = nil` to dismiss the keyboard
- Add a toolbar Done button: `.toolbar { ToolbarItemGroup(placement: .keyboard) { … } }`
- Validate in `onChange(of:)` and show errors under the field

QUICK SUMMARY

- `TextField` / `SecureField` / `TextEditor`
- `@FocusState` + `.focused`
- `onSubmit` + keyboard modifiers

Forms

BUILDING FORMS

`Form` arranges controls into platform-standard settings layouts: grouped rows on iOS, aligned label columns on macOS. Split it into `Sections` with headers and footers. `LabeledContent` shows a label with a value or custom control. Disable the submit button until the input is valid.

NOTES

- Controls inside a Form adapt their style automatically
- Section footers are good for help text and errors
- `.formStyle(.grouped)` gives iOS-style grouping on macOS
- Navigation links inside forms push detail pages

A SIGN-UP FORM

```
struct SignUpForm: View {
    @State private var name = ""
    @State private var email = ""
    @State private var plan = Plan.free
    @State private var agreed = false
    private var isValid: Bool {
        !name.isEmpty && email.contains("@") && agreed
    }

    var body: some View {
        Form {
            Section("Account") {
                TextField("Name", text: $name)
                TextField("Email", text: $email)
            }
            Section {
                Picker("Plan", selection: $plan) { /* options */ }
                LabeledContent("Price", value: plan.price,
                               format: .currency(code: "USD"))
            } footer: { Text("Cancel anytime.") }
            Toggle("I agree to the terms", isOn: $agreed)
            Button("Create Account", action: submit).disabled(!isValid)
        }
    }
}
```

CODE EXPLAINED

- → `Form` — platform settings layout
- → `Section("Title")` — grouped rows with header
- → `footer:` — help text under a section
- → `LabeledContent` — label + value row
- → `.disabled(!isValid)` — gate submission

MINIMUM OS

- `Form`, `Section`: iOS 13
- `LabeledContent`, `.formStyle`: iOS 16

FORM ROWS

VIEW	SHOWS
`TextField`	Editable text
`Toggle / Picker`	Settings
`LabeledContent`	Read-only value
`NavigationLink`	Push sub-page
`Button`	Action row

UIKIT / REACT EQUIVALENT

- **`UIKit`** Static `UITableView` with grouped style and custom cells
- **`React Native`** `ScrollView` of inputs; react-hook-form for validation

⚠ COMMON MISTAKES

- Validating only on submit: show problems as the user types
- Putting long scrolling content in a Form: use List or ScrollView
- Custom HStack rows that break macOS alignment: use `LabeledContent`

✓ TIPS & BEST PRACTICES

- Keep a computed `isValid` in the view or model
- Use `.alert` to report server errors after submit
- Use `@FocusState` to jump to the first invalid field

QUICK SUMMARY

- `Form` + `Section`
- `LabeledContent` for value rows
- disable submit until valid

Gestures

TOUCH GESTURES

Attach gestures with `.gesture(...)` or shortcut modifiers like `.onTapGesture`. `DragGesture`, `LongPressGesture`, `MagnifyGesture` and `RotateGesture` report values as they change. `@GestureState` holds a value only **during** the gesture and resets automatically; combine gestures with `simultaneously`, `sequenced` and `exclusively`.

NOTES

- Prefer `Button` over `onTapGesture` for actions
- `.highPriorityGesture` wins over children's gestures
- `.contentShape(.rect)` makes empty areas tappable
- Gesture values: `translation`, `magnification`, `rotation`

DRAG & MAGNIFY

```
struct DraggableCard: View {
    @State private var offset = CGSize.zero          // where it rests
    @GestureState private var drag = CGSize.zero     // during drag
    @State private var scale = 1.0

    var body: some View {
        RoundedRectangle(cornerRadius: 20).fill(.blue)
            .frame(width: 200, height: 120)
            .scaleEffect(scale)
            .offset(x: offset.width + drag.width, y: offset.height + drag.height)
            .gesture(
                DragGesture()
                    .updating($drag) { v, state, _ in state = v.translation }
                    .onEnded { v in
                        offset.width += v.translation.width
                        offset.height += v.translation.height
                    }
                    .simultaneously(with: MagnifyGesture()
                        .onChanged { scale = $0.magnification })
            )
            .onTapGesture(count: 2) { withAnimation { scale = 1 } }
    }
}
```

CODE EXPLAINED

- → `@GestureState` — resets when the gesture ends
- → `.updating($drag)` — live value
- → `.onEnded` — commit final position
- → `.simultaneously(with:)` — combine gestures
- → `onTapGesture(count: 2)` — double tap

GESTURE TYPES

GESTURE	VALUE
`TapGesture(count:)`	—
`LongPressGesture`	Bool
`DragGesture`	translation, location
`MagnifyGesture`	magnification
`RotateGesture`	rotation (Angle)

MINIMUM OS

- Drag / LongPress / Tap gestures: iOS 13
- `MagnifyGesture`, `RotateGesture`: iOS 17 (were `MagnificationGesture`)

UIKIT / REACT EQUIVALENT

- **UIKit** `UIPanGestureRecognizer`, `UIPinchGestureRecognizer`
- **React Native** react-native-gesture-handler + Reanimated

ACCESSIBILITY

- ♿ Provide an `.accessibilityAction` for anything only reachable by a gesture
- ♿ Custom drag UIs need an alternative like buttons or a slider

⚠ COMMON MISTAKES

- A gesture on a ScrollView child blocking scrolling: use `simultaneousGesture`
- Storing drag offsets in `@State` and forgetting to reset them
- Gestures on views with transparent areas that don't receive touches

✓ TIPS & BEST PRACTICES

- Animate the snap-back in `onEnded` with a spring
- Use `minimumDistance:` on drags inside scroll views
- `.sensoryFeedback` adds haptics at gesture milestones

QUICK SUMMARY

- `.gesture(DragGesture()...)`
- `@GestureState` for transient values
- compose with `simultaneously` etc.

Lists

LISTS OF DATA

`List` shows scrolling rows in the platform's standard style, with built-in separators, sections, swipe actions and selection. Feed it a collection of **Identifiable** items (or give an `id:` key path) so SwiftUI can track rows across changes. `ForEach` inside a `List` lets you mix static rows, sections and dynamic data.

NOTES

- List is lazy: rows are created as they scroll on screen
- IDs must be stable and unique, never array indices
- `listStyle` changes the whole look (`.insetGrouped` , `.plain`)
- Row modifiers: `listRowSeparator` , `listRowBackground` , `listRowInsets`
- Swift set: 071 · Common Standard Protocols

LIST & FOREACH

```
struct Contact: Identifiable {
    let id: UUID
    var name: String
    var isFavorite: Bool
}

List {
    Section("Favorites") {
        ForEach(contacts.filter(\.isFavorite)) { c in
            Label(c.name, systemImage: "star.fill")
        }
    }
    Section("All") {
        ForEach(contacts) { c in
            NavigationLink(c.name, value: c.id)
        }
    }
}
.listStyle(.insetGrouped)

List(tags, id: \.self) { tag in Text(tag) }    // Strings, id by self

#Preview { NavigationStack { ContactList() } }
```

CODE EXPLAINED

- → `Identifiable` — stable `id` per row
- → `Section("…")` — grouped headers
- → `ForEach` — rows from data inside List
- → `id: \.self` — for simple unique values
- → `.listStyle` — whole-list appearance

MINIMUM OS

- `List` , `ForEach` : iOS 13
- `listRowSeparator` , `.insetGrouped` on iPhone: iOS 15

UIKIT / REACT EQUIVALENT

- **UIKit** `UITableView` / `UICollectionView` list + diffable data source
- **React Native** `<FlatList data keyExtractor>` / `<SectionList>`

LIST STYLES

STYLE	LOOK
`.automatic`	Platform default
`.insetGrouped`	Rounded grouped cards
`.plain`	Edge-to-edge rows
`.sidebar`	Collapsible sections
`.grouped`	Full-width grouped

ACCESSIBILITY

- ♿ Rows combine their text for VoiceOver automatically
- ♿ Add `.accessibilityElement(children: .combine)` to custom multi-part rows

⚠ COMMON MISTAKES

- `id: \.self` on items that can repeat or change: rows glitch
- Using array indices as IDs: insert/delete animations break
- Putting a List inside a ScrollView: it collapses to zero height

✓ TIPS & BEST PRACTICES

- Make model types `Identifiable` once and reuse everywhere
- Use `.listRowSeparator(.hidden)` for custom row designs
- Use `ContentUnavailableView` when the list is empty (Sheet 29)

QUICK SUMMARY

- `List` + `ForEach` + `Section`
- stable `Identifiable` IDs
- `.listStyle` for appearance

List Editing & Selection

EDITING & SELECTING ROWS

Lists support editing out of the box: `.onDelete` and `.onMove` on a `ForEach` enable swipe-to-delete and drag-to-reorder, and `EditButton` toggles edit mode. `swipeActions` add custom leading or trailing buttons. Pass a `selection` binding to make rows selectable, including multiple selection in edit mode.

NOTES

- `.onDelete` gives an `IndexSet`; `.onMove` gives `IndexSet` + destination
- Custom swipe actions replace the default delete
- Selection uses each row's ID type
- `contextMenu` adds long-press menus

DELETE, MOVE, SWIPE, SELECT

```
@State private var todos: [Todo] = Todo.samples
@State private var selection = Set<Todo.ID>()

NavigationStack {
    List(selection: $selection) {
        ForEach(todos) { todo in
            Text(todo.title)
                .swipeActions(edge: .leading) {
                    Button("Done") { complete(todo) }.tint(.green)
                }
                .contextMenu { Button("Duplicate") { dup(todo) } }
        }
        .onDelete { todos.remove(atOffsets: $0) }
        .onMove { todos.move(fromOffsets: $0, toOffset: $1) }
    }
    .toolbar { EditButton() }
    .navigationTitle("\(selection.count) selected")
}
```

CODE EXPLAINED

- → `.onDelete` — swipe to delete
- → `.onMove` — drag to reorder in edit mode
- → `swipeActions(edge:)` — custom swipe buttons
- → `List(selection:)` — selectable rows
- → `EditButton()` — toggles edit mode

MINIMUM OS

- `onDelete`, `onMove`, `EditButton`: iOS 13
- `swipeActions`: iOS 15

UIKIT / REACT EQUIVALENT

- **UIKit** `tableView(_:commit:forRowAt:)`, `UISwipeActionsConfiguration`
- **React Native** react-native-swipe-list-view, draggable-flatlist

ACCESSIBILITY

- ♿ Swipe actions appear in VoiceOver's actions rotor automatically
- ♿ Custom drag reordering needs `.accessibilityAction` alternatives

EDITING APIS

API	GIVES YOU
`.onDelete { offsets }`	IndexSet
`.onMove { from, to }`	IndexSet, Int
`remove(atOffsets:)`	Array helper
`move(fromOffsets:toOffset:)`	Array helper
`.deleteDisabled(cond)`	Block per row

⚠ COMMON MISTAKES

- Applying `.onDelete` to `List` instead of the `ForEach`
- Filtering with `.filter` then using offsets on the original array: wrong rows deleted
- Destructive swipe action without `role: .destructive`

✓ TIPS & BEST PRACTICES

- Delete by ID when the list is filtered: map offsets to items first
- Use `allowsFullSwipe: false` for risky actions
- SwiftData lists: delete via `modelContext.delete` (Sheet 42)

QUICK SUMMARY

- `onDelete` / `onMove` on ForEach
- `swipeActions` / `contextMenu`
- `List(selection:)` + EditButton

ScrollView & Lazy Stacks

CUSTOM SCROLLING

When you need custom layouts instead of List rows, use `ScrollView` with a `LazyVStack` or `LazyHStack`, which create children only when needed. Scroll **target behavior** adds paging and snapping, `scrollPosition` reads and sets the visible item, and `ScrollViewReader` scrolls to an ID on older systems.

NOTES

- Plain `VStack` in a ScrollView builds everything up front
- `.scrollTargetLayout()` marks which children are snap targets
- `.scrollIndicators(.hidden)` hides scroll bars
- `.scrollClipDisabled()` lets shadows overflow

SCROLLVIEW PATTERNS

```
ScrollView {
    LazyVStack(spacing: 12, pinnedViews: .sectionHeaders) {
        Section { ForEach(posts) { PostCard($0) } }
        header: { Text("Today").font(.headline) }
    }
    .padding()
}

@State private var current: Card.ID?

ScrollView(.horizontal) {
    LazyHStack(spacing: 16) {
        ForEach(cards) { CardView($0).containerRelativeFrame(.horizontal) }
    }
    .scrollTargetLayout()
}
.scrollTargetBehavior(.viewAligned)            // snap to cards
.scrollPosition(id: $current)                  // read/set visible card
.scrollIndicators(.hidden)

Button("Back to first") { withAnimation { current = cards.first?.id } }
```

CODE EXPLAINED

- → `LazyVStack` — lazy rows in a ScrollView
- → `pinnedViews: .sectionHeaders` — sticky headers
- → `scrollTargetBehavior(.viewAligned)` — snapping
- → `scrollPosition(id:)` — two-way scroll binding
- → `scrollTargetLayout()` — snap targets

MINIMUM OS

- `ScrollViewReader`, lazy stacks: iOS 14
- `scrollPosition(id:)`, `scrollTargetBehavior`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UIScrollView` / compositional layout with `orthogonalScrollingBehavior`
- **React Native** `<ScrollView>` / `<FlatList horizontal pagingEnabled>`

SCROLL MODIFIERS

MODIFIER	EFFECT
`.scrollTargetBehavior(.paging)`	Full-page paging
`.scrollDismissesKeyboard(.interactively)`	Drag hides keyboard
`.defaultScrollAnchor(.bottom)`	Start at bottom (chat)
`.contentMargins(16)`	Inset content
`.scrollBounceBehavior(.basedOnSize)`	No bounce when it fits

⚠ COMMON MISTAKES

- `VStack` with 1,000 items in a ScrollView: slow first render
- Forgetting `.scrollTargetLayout()`: snapping does nothing
- Nesting a vertical List inside a vertical ScrollView

✓ TIPS & BEST PRACTICES

- Use `.defaultScrollAnchor(.bottom)` for chat screens
- Use `ScrollViewReader` + `proxy.scrollTo(id)` if you support iOS 16
- Use `.onScrollGeometryChange` (iOS 18) for scroll-driven effects

QUICK SUMMARY

- ScrollView + lazy stacks
- snapping via target behavior
- `scrollPosition(id:)` to read/set

Search & Refresh

SEARCH, REFRESH & EMPTY STATES

`.searchable` adds a system search field bound to a String; filter your data from it, optionally with **scopes** and **suggestions**. `.refreshable` adds pull-to-refresh to Lists and ScrollViews and awaits your async function. `ContentUnavailableView` shows standard empty and no-results screens.

NOTES

- Attach `.searchable` inside a NavigationStack for the bar placement
- `.refreshable` shows the spinner until your `async` work finishes
- `ContentUnavailableView.search` adapts to the query text
- Debounce server searches with `.task(id: query)`

SEARCH & REFRESH

```
@State private var query = ""
@State private var scope = Scope.all

var filtered: [Book] {
    books.filter { query.isEmpty || $0.title.localizedStandardContains(query) }
}

NavigationStack {
    List(filtered) { BookRow($0) }
        .overlay {
            if filtered.isEmpty { ContentUnavailableView.search(text: query) }
        }
        .searchable(text: $query, prompt: "Title or author")
        .searchScopes($scope) {
            ForEach(Scope.allCases) { Text($0.label).tag($0) }
        }
        .refreshable { await store.reload() }
        .task(id: query) {                          // debounced server search
            try? await Task.sleep(for: .milliseconds(300))
            await store.search(query)
        }
}
```

CODE EXPLAINED

- → `.searchable(text:)` — search field
- → `localizedStandardContains` — user-friendly match
- → `ContentUnavailableView.search` — no results
- → `.searchScopes` — scope bar
- → `.refreshable` — pull to refresh

MINIMUM OS

- `searchable`, `refreshable`: iOS 15
- `searchScopes`: iOS 16.4; `ContentUnavailableView`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UISearchController` + `UIRefreshControl`
- **React Native** `TextInput` header + `RefreshControl` on FlatList

RELATED MODIFIERS

MODIFIER	DOES
`.searchSuggestions { }`	Suggestion rows
`.searchCompletion(_:)`	Fill query from a suggestion
`\.isSearching`	Environment flag
`.searchPresentationToolbarBehavior`	Keep title visible

⚠ COMMON MISTAKES

- Calling the server on every keystroke: debounce with `.task(id:)`
- Blank screen when nothing matches: show `ContentUnavailableView`
- Refresh that returns before the work finishes (not awaited)

✓ TIPS & BEST PRACTICES

- Use `ContentUnavailableView("No Notes", systemImage:, description:)` for empty states
- Keep filtering in a computed property or the model
- Add `.searchable(…, placement: .navigationBarDrawer(displayMode: .always))` to pin it

QUICK SUMMARY

- `.searchable` + filter
- `.refreshable` awaits work
- `ContentUnavailableView` for empty

NavigationStack

STACK NAVIGATION

`NavigationStack` manages a push/pop stack of screens. `NavigationLink(value:)` pushes a **value**, and `navigationDestination(for:)` declares which view to show for values of that type, keeping links and destinations decoupled. Titles and bar display mode are set on the content inside the stack.

NOTES

- Put `navigationDestination` inside the stack, not on lazy rows
- Link values must be `Hashable`
- Older `NavigationView` and `NavigationLink(destination:)` are deprecated patterns
- Back button and swipe-back come for free
- Swift set: 071 · Common Standard Protocols

NAVIGATIONSTACK

```
struct LibraryView: View {
    let books: [Book]
    var body: some View {
        NavigationStack {
            List(books) { book in
                NavigationLink(value: book) {          // push a value
                    BookRow(book)
                }
            }
            .navigationTitle("Library")
            .navigationDestination(for: Book.self) { book in
                BookDetail(book: book)
            }
            .navigationDestination(for: Author.self) { AuthorView($0) }
        }
    }
}

struct BookDetail: View {
    let book: Book
    var body: some View {
        Form { NavigationLink(book.author.name, value: book.author) }
            .navigationTitle(book.title)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

CODE EXPLAINED

- → `NavigationStack` — push/pop container
- → `NavigationLink(value:)` — push data, not a view
- → `navigationDestination(for:)` — map type → view
- → `navigationTitle` — set on content
- → `.inline` title mode — small title

MINIMUM OS

- `NavigationStack`, `navigationDestination`: iOS 16
- Before iOS 16: `NavigationView` + `NavigationLink(destination:)`

TITLE MODES

MODE	LOOK
`.automatic`	Inherit
`.large`	Large title, collapses on scroll
`.inline`	Small centered title

UIKIT / REACT EQUIVALENT

- **UIKit** `UINavigationController.pushViewController`
- **React Navigation** `createNativeStackNavigator` + `navigation.navigate`

⚠ COMMON MISTAKES

- Nesting a NavigationStack inside another: double bars, broken pushes
- Putting `navigationDestination` on a row inside a lazy List: may not register
- Using non-Hashable values in `NavigationLink(value:)`

✓ TIPS & BEST PRACTICES

- One NavigationStack per tab, at the root of each tab
- Use an enum `Route: Hashable` for all destinations (Sheet 31)
- `.toolbarTitleMenu { }` adds a menu under the title

QUICK SUMMARY

- `NavigationStack` + value links
- `navigationDestination(for:)`
- titles on content

Programmatic Navigation

NAVIGATING FROM CODE

Bind a `NavigationStack` to a `path` and you control navigation with plain data: append to push, remove to pop, set to `[]` to go to the root. A typed array (`[Route]`) works when all destinations share one enum; `NavigationPath` holds mixed types. A small **router** model makes deep links and restoration easy.

NOTES

- The path is your navigation state: save and restore it
- `NavigationPath` is type-erased and can be Codable
- Deep links: parse a URL into routes, then set the path
- Keep the router in the environment
- Swift set: O64 · Enums — Associated Values

ROUTER + PATH

```
enum Route: Hashable {
    case book(Book.ID), author(Author.ID), settings
}

@MainActor @Observable final class Router {
    var path: [Route] = []
    func open(_ r: Route) { path.append(r) }
    func popToRoot() { path.removeAll() }
}

struct RootView: View {
    @State private var router = Router()
    var body: some View {
        NavigationStack(path: $router.path) {
            HomeView()
                .navigationDestination(for: Route.self) { route in
                    switch route {
                    case .book(let id):   BookDetail(id: id)
                    case .author(let id): AuthorView(id: id)
                    case .settings:       SettingsView()
                    }
                }
        }
        .environment(router)
        .onOpenURL { url in router.path = Route.parse(url) }
    }
}
```

CODE EXPLAINED

- → `NavigationStack(path:)` — bound navigation state
- → `enum Route: Hashable` — every destination
- → `path.append` / `removeAll` — push / pop to root
- → `.environment(router)` — any view can navigate
- → `.onOpenURL` — deep link into the path

MINIMUM OS

- `NavigationStack(path:)`, `NavigationPath`: iOS 16
- `@Observable` router: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** Coordinators pushing view controllers
- **React Navigation** `navigation.reset({ routes })` + linking config

PATH OPERATIONS

CODE	EFFECT
`path.append(r)`	Push
`path.removeLast()`	Pop
`path.removeLast(path.count)`	Pop to root
`path = [a, b]`	Jump to a deep stack

⚠ COMMON MISTAKES

- Using `NavigationLink(destination:)` and path state together: they don't sync
- Storing whole model objects in routes: store IDs
- Separate path state per tab forgotten: tabs share one stack

✓ TIPS & BEST PRACTICES

- Encode `NavigationPath.codable` to restore navigation on relaunch
- Unit-test URL → `[Route]` parsing without UI
- One router per tab, held by the tab's root view

QUICK SUMMARY

- path binding = navigation state
- `enum Route` + switch destination
- router model for deep links

TabView

TABS

`TabView` shows top-level sections with a tab bar (or sidebar). Since iOS 18 you declare tabs with the `Tab` type, which supports a `selection` value, badges, and the `.sidebarAdaptable` style that becomes a sidebar on iPad. With `.tabViewStyle(.page)` the same view becomes swipeable pages.

NOTES

- Each tab usually has its own NavigationStack
- Bind `selection` to switch tabs from code
- `Tab(role: .search)` gets the system search tab
- On iOS 26 the tab bar adopts Liquid Glass and can minimize on scroll

TABVIEW

```
enum AppTab: Hashable { case home, inbox, profile, search }
@State private var tab = AppTab.home

TabView(selection: $tab) {
    Tab("Home", systemImage: "house", value: .home) {
        NavigationStack { HomeView() }
    }
    Tab("Inbox", systemImage: "tray", value: .inbox) {
        NavigationStack { InboxView() }
    }
    .badge(unreadCount)
    Tab("Profile", systemImage: "person", value: .profile) {
        NavigationStack { ProfileView() }
    }
    Tab(value: .search, role: .search) {
        NavigationStack { SearchView() }
    }
}
.tabViewStyle(.sidebarAdaptable)                    // sidebar on iPad

TabView { ForEach(pages) { OnboardingPage($0) } }
    .tabViewStyle(.page)                            // swipe pages
```

CODE EXPLAINED

- → `Tab(_:systemImage:value:)` — declarative tab
- → `TabView(selection:)` — switch from code
- → `.badge` — count on the tab
- → `role: .search` — system search tab
- → `.page` style — onboarding pager

MINIMUM OS

- `Tab` type, `.sidebarAdaptable`, `role: .search`: iOS 18
- Before iOS 18: `.tabItem { Label(…) }` + `.tag`

UIKIT / REACT EQUIVALENT

- **UIKit** `UITabBarController` with view controllers
- **React Navigation** `createBottomTabNavigator`

TABVIEW STYLES

STYLE	LOOK
`.automatic`	Tab bar
`.sidebarAdaptable`	Tabs ↔ sidebar (iPad)
`.tabBarOnly`	Never a sidebar
`.page`	Swipeable pages

⚠ COMMON MISTAKES

- One NavigationStack wrapping the whole TabView: tab bar disappears on push
- More than 5 tabs on iPhone: extras go to a "More" tab
- Mixing `Tab` and `.tabItem` styles in one TabView

✓ TIPS & BEST PRACTICES

- Keep the selected tab in `@SceneStorage` to restore it
- Use `.tabBarMinimizeBehavior(.onScrollDown)` on iOS 26
- Use `TabSection` to group tabs in the iPad sidebar

QUICK SUMMARY

- `TabView` + `Tab` (iOS 18)
- stack per tab
- `.sidebarAdaptable` / `.page` styles

NavigationSplitView

SPLIT VIEWS

`NavigationSplitView` builds multi-column layouts for iPad, Mac and visionOS: a **sidebar**, an optional **content** column and a **detail** column. On iPhone it collapses into a single navigation stack automatically. Drive it with selection bindings so the detail column shows whatever is selected.

NOTES

- Two columns: sidebar + detail; three: sidebar + content + detail
- `columnVisibility` controls which columns show
- Selection in a List drives the next column
- Put a NavigationStack inside detail for deeper pushes

NAVIGATIONSPLITVIEW

```
@State private var folder: Folder.ID?
@State private var note: Note.ID?
@State private var visibility = NavigationSplitViewVisibility.all

NavigationSplitView(columnVisibility: $visibility) {
    List(folders, selection: $folder) { f in
        Label(f.name, systemImage: "folder")
    }
    .navigationTitle("Folders")
} content: {
    if let folder {
        List(notes(in: folder), selection: $note) { Text($0.title) }
    } else { Text("Pick a folder") }
} detail: {
    if let note { NoteEditor(id: note) }
    else { ContentUnavailableView("No Note", systemImage: "note.text") }
}
.navigationSplitViewStyle(.balanced)
```

CODE EXPLAINED

- → sidebar / content / detail — three columns
- → `List(selection:)` — selection drives next column
- → `columnVisibility` — show/hide columns
- → `.balanced` / `.prominentDetail` — column style
- → collapses to a stack on iPhone

MINIMUM OS

- `NavigationSplitView`: iOS 16 / macOS 13
- `.inspector` side panel: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UISplitViewController` (double/triple column)
- **React Navigation** Drawer navigator + custom tablet layout

VISIBILITY VALUES

VALUE	SHOWS
`.all`	Every column
`.doubleColumn`	Content + detail
`.detailOnly`	Detail only
`.automatic`	System decides

⚠ COMMON MISTAKES

- Using split view for an iPhone-only app: TabView/Stack is simpler
- Not handling `nil` selection in detail: blank column
- Pushing in the sidebar instead of changing selection

✓ TIPS & BEST PRACTICES

- Test in iPad portrait, landscape and Slide Over
- Set column widths with `navigationSplitViewColumnWidth`
- Combine with `TabView(.sidebarAdaptable)` for top-level sections

QUICK SUMMARY

- sidebar → content → detail
- selection bindings drive columns
- auto-collapses on iPhone

Sheets & Full-Screen Covers

MODAL SHEETS

`.sheet` presents a view modally, controlled by a Bool (`isPresented:`) or an optional Identifiable **item** (`item:`), which also passes the data to show. `fullScreenCover` covers the whole screen. **Detents** make half-height sheets, and the presented view closes itself with `@Environment(\.dismiss)`.

NOTES

- Prefer `item:` when the sheet needs data: no stale values
- Each sheet gets its own environment; re-inject models if needed
- `interactiveDismissDisabled` blocks swipe-down
- Sheets on iPad show as centered cards

SHEETS & DETENTS

```
@State private var showNew = false
@State private var editing: Note?              // item drives sheet

List(notes) { note in
    Button(note.title) { editing = note }
}
.toolbar { Button("New", systemImage: "plus") { showNew = true } }
.sheet(isPresented: $showNew) {
    NewNoteView()
        .presentationDetents([.medium, .large])
        .presentationDragIndicator(.visible)
}
.sheet(item: $editing) { note in
    EditNoteView(note: note)
        .interactiveDismissDisabled(note.hasUnsavedChanges)
}
.fullScreenCover(isPresented: $showOnboarding) { OnboardingView() }

struct NewNoteView: View {
    @Environment(\.dismiss) private var dismiss
    var body: some View { Button("Cancel") { dismiss() } }
}
```

CODE EXPLAINED

- → `.sheet(isPresented:)` — Bool-driven
- → `.sheet(item:)` — data-driven, auto-nil on close
- → `presentationDetents` — half/full heights
- → `fullScreenCover` — covers everything
- → `dismiss()` — close from inside

MINIMUM OS

- `.sheet`, `fullScreenCover`: iOS 13–14
- `presentationDetents`: iOS 16

PRESENTATION MODIFIERS

MODIFIER	EFFECT
`.presentationDetents([.medium])`	Heights
`.presentationBackground(.thinMaterial)`	Background
`.presentationCornerRadius(24)`	Corners
`.presentationBackgroundInteraction(.enabled)`	Use view behind
`.presentationSizing(.form)`	iPad size (iOS 18)

UIKIT / REACT EQUIVALENT

- **UIKit** `present(_:animated:)` + `UISheetPresentationController`
- **React Navigation** `presentation: 'modal'` screen / RN `<Modal>`

⚠ COMMON MISTAKES

- Bool sheet reading a separately stored selected item: shows stale data; use `item:`
- Several `.sheet` modifiers on the same view for different Bools: use an enum item
- Forgetting environment models inside the sheet content

✓ TIPS & BEST PRACTICES

- Use an `enum SheetKind: Identifiable` with one `.sheet(item:)`
- Wrap sheet content in a NavigationStack to get a toolbar
- Use `.presentationDetents([.height(200)])` for small pickers

QUICK SUMMARY

- `.sheet(isPresented:)` / `.sheet(item:)`
- detents for half sheets
- `dismiss()` from inside

Alerts & Dialogs

ALERTS & CONFIRMATIONS

`.alert` shows a short message with buttons; `.confirmationDialog` shows an action sheet of choices, usually to confirm something destructive. Both are driven by a Bool and can take **presenting data** so the buttons know which item they act on. Button roles (`.destructive`, `.cancel`) place and color buttons correctly.

NOTES

- A Cancel button is added automatically if you don't provide one
- `presenting:` passes the item into the actions and message
- Alerts can contain `TextField`s
- `LocalizedError` makes error alerts easy

ALERT & DIALOG

```
@State private var toDelete: Note?
@State private var showConfirm = false
@State private var error: AppError?

List(notes) { note in
    Text(note.title)
        .swipeActions {
            Button("Delete", role: .destructive) {
                toDelete = note; showConfirm = true
            }
        }
}
.confirmationDialog("Delete note?", isPresented: $showConfirm,
                    titleVisibility: .visible, presenting: toDelete) { note in
    Button("Delete \"\(note.title)\"", role: .destructive) { delete(note) }
    Button("Cancel", role: .cancel) { }
}
.alert(isPresented: .constant(error != nil), error: error) { _ in
    Button("OK") { error = nil }
} message: { e in Text(e.recoverySuggestion ?? "") }
```

CODE EXPLAINED

- → `.confirmationDialog(presenting:)` — item-aware choices
- → `role: .destructive` / `.cancel` — placement + color
- → `titleVisibility: .visible` — show the title
- → `.alert(error:)` — alert from a `LocalizedError`
- → `message:` — body text

MINIMUM OS

- `.alert` / `.confirmationDialog` with actions: iOS 15
- `TextField` inside alerts: iOS 16

UIKIT / REACT EQUIVALENT

- **UIKit** `UIAlertController` (.alert / .actionSheet)
- **React Native** `Alert.alert(title, msg, buttons)` / ActionSheetIOS

WHICH ONE

NEED	USE
Inform / error	.alert
Confirm destructive action	.confirmationDialog
Choose among actions	.confirmationDialog
Ask for a short value	.alert + TextField

⚠ COMMON MISTAKES

- Putting long content in an alert: use a sheet
- Reading a separately stored item in alert buttons: pass it with `presenting:`
- Showing alerts for every validation error: inline messages are better

✓ TIPS & BEST PRACTICES

- Make your errors `LocalizedError` to use `.alert(isPresented:error:)`
- Name the destructive button after the action: "Delete Note", not "OK"
- On iPad, dialogs show as popovers from the source view

QUICK SUMMARY

- `.alert` for messages
- `.confirmationDialog` for choices
- roles + `presenting:` data

Toolbars, Menus & Popovers

TOOLBARS, MENUS & POPOVERS

`.toolbar` places items in the navigation bar, bottom bar or keyboard accessory using **placements**. Menu shows a button that opens a list of actions, `.contextMenu` adds a long-press menu, `.popover` shows content attached to a view (a sheet on iPhone unless told otherwise), and `.inspector` adds a trailing side panel.

NOTES

- Placements are semantic: the system positions them per platform
- `ToolbarItemGroup` places several items together
- `Menu` can nest menus, pickers and dividers
- `.presentationCompactAdaptation(.popover)` keeps popovers on iPhone

TOOLBAR, MENU, POPOVER

```
.toolbar {
    ToolbarItem(placement: .topBarLeading) { EditButton() }
    ToolbarItem(placement: .primaryAction) {
        Menu("Sort", systemImage: "arrow.up.arrow.down") {
            Picker("Sort", selection: $sort) {
                Text("Date").tag(Sort.date)
                Text("Title").tag(Sort.title)
            }
            Divider()
            Button("Export…", systemImage: "square.and.arrow.up", action: export)
        }
    }
    ToolbarItemGroup(placement: .bottomBar) {
        Spacer(); Text("\(count) notes").font(.caption); Spacer()
    }
}

Image(systemName: "info.circle")
    .onTapGesture { showTip = true }
    .popover(isPresented: $showTip) {
        Text("Tip text").padding()
            .presentationCompactAdaptation(.popover)
    }
```

CODE EXPLAINED

- → `ToolbarItem(placement:)` — position by meaning
- → `Menu` — button that opens actions
- → `Picker` inside Menu — checkmarked choices
- → `ToolbarItemGroup(.bottomBar)` — bottom toolbar
- → `.popover` + compact adaptation — bubble on iPhone

MINIMUM OS

- `.toolbar`, `Menu`: iOS 14
- `.inspector`: iOS 17; `presentationCompactAdaptation`: iOS 16.4

UIKIT / REACT EQUIVALENT

- **UIKit** `navigationItem.rightBarButtonItems`, `UIMenu`, `UIPopoverPresentationController`
- **React Navigation** `headerRight` option; menu libraries

COMMON PLACEMENTS

PLACEMENT	WHERE (IOS)
`.topBarLeading / .topBarTrailing`	Nav bar sides
`.primaryAction`	Main action spot
`.principal`	Title area
`.bottomBar`	Bottom toolbar
`.keyboard`	Above the keyboard
`.confirmationAction / .cancellationAction`	Sheet buttons

ACCESSIBILITY

- ♿ Icon-only toolbar items need labels: `Button("Sort", systemImage:)` provides one
- ♿ Context menus must have a visible alternative; long press isn't discoverable

⚠ COMMON MISTAKES

- Hard-coding `.topBarTrailing` everywhere: macOS and iPad place things differently
- Hiding important actions only in a context menu
- Expecting `.popover` to show as a bubble on iPhone by default

✓ TIPS & BEST PRACTICES

- Use `.confirmationAction` / `.cancellationAction` in sheets
- Use `.toolbar(.hidden, for: .tabBar)` to hide bars on a screen
- `ControlGroup` groups related toolbar buttons

QUICK SUMMARY

- `.toolbar` with semantic placements
- `Menu` / `.contextMenu`
- `.popover` / `.inspector`

.task, onAppear & onChange

VIEW LIFECYCLE HOOKS

`.task` starts async work when a view appears and **cancels it automatically** when the view disappears; `.task(id:)` also restarts when a value changes. `onAppear` and `onDisappear` run synchronous code, and `onChange(of:)` reacts when a value changes. These replace UIKit's `viewDidLoad`/`viewWillAppear` style hooks.

NOTES

- `.task` runs on the main actor by default (in a View)
- `onAppear` can fire more than once (tabs, navigation)
- `onChange` closure gets old and new values
- Use `initial: true` to also run once at appearance
- Swift set: 098 · async / await, 100 · Task Cancellation

.TASK, ONAPPEAR, ONCHANGE

```
struct ArticleView: View {
    let id: Article.ID
    @State private var article: Article?
    @State private var fontSize = 17.0

    var body: some View {
        ScrollView { if let article { ArticleBody(article) } }
            .task(id: id) {                          // restarts when id changes
                article = try? await api.article(id)
            }
            .onAppear { analytics.log("article_open") }
            .onDisappear { saveReadingPosition() }
            .onChange(of: fontSize) { old, new in
                prefs.save(fontSize: new)
            }
            .onChange(of: article?.isRead, initial: true) {
                updateBadge()
            }
    }
}
```

CODE EXPLAINED

- → `.task(id:)` — async, cancels/restarts automatically
- → `onAppear` / `onDisappear` — sync hooks
- → `onChange(of:) { old, new in }` — react to changes
- → `initial: true` — run once at start too

MINIMUM OS

- `.task`, `.task(id:)`: iOS 15
- Two-value `onChange(of:)` and `initial:`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `viewDidLoad`, `viewWillAppear`, `viewDidDisappear`, KVO
- **React Native** `useEffect(() => {…; return cleanup}, [id])`

HOOK CHOICE

NEED	USE
Load data on appear	.task
Reload when input changes	.task(id:)
Log / start a timer	onAppear
Save when leaving	onDisappear
React to a value	onChange(of:)

⚠ COMMON MISTAKES

- Starting `Task { }` in `onAppear`: no automatic cancellation
- Expecting `onAppear` to run only once
- Old one-argument `onChange` closures: deprecated in iOS 17

✓ TIPS & BEST PRACTICES

- Use `.task(id:)` + `Task.sleep` to debounce input
- Keep loading logic in the model; `.task { await model.load() }`
- Guard against reloading data you already have

QUICK SUMMARY

- `.task` — async + auto-cancel
- `onAppear` / `onDisappear`
- `onChange(of:)` for reactions

Loading & Error States

LOADING & ERROR STATES

Real screens are loading, loaded, empty or failed. Model this with an **enum** in a `@MainActor` model and `switch` over it in the view, so every state has a deliberate UI. `ProgressView` shows activity, `ContentUnavailableView` handles empty and error states, and `.redacted(reason: .placeholder)` shows a skeleton of the real layout.

NOTES

- One enum prevents impossible combinations (loading + error)
- Keep previous data visible during refresh if possible
- Treat `CancellationError` as not-an-error
- Offer a retry action on failure
- Swift set: 064 · Enums — Associated Values, 101 · Actors & @MainActor

STATE ENUM PATTERN

```
@MainActor @Observable final class FeedModel {
    enum Phase { case loading, loaded([Post]), empty, failed(Error) }
    var phase = Phase.loading
    func load() async {
        do {
            let posts = try await api.posts()
            phase = posts.isEmpty ? .empty : .loaded(posts)
        } catch is CancellationError {} catch { phase = .failed(error) }
    }
}

struct FeedView: View {
    @State private var model = FeedModel()
    var body: some View {
        Group {
            switch model.phase {
            case .loading:
                PostList(posts: Post.placeholders)
                    .redacted(reason: .placeholder)
            case .loaded(let posts): PostList(posts: posts)
            case .empty:
                ContentUnavailableView("No Posts", systemImage: "tray")
            case .failed:
                ContentUnavailableView {
                    Label("Couldn't Load", systemImage: "wifi.slash")
                } actions: {
                    Button("Retry") { Task { await model.load() } }
                }
            }
        }
        .task { await model.load() }
    }
}
```

CODE EXPLAINED

- → `enum Phase` — every screen state
- → `catch is CancellationError` — ignore cancels
- → `.redacted(reason: .placeholder)` — skeleton
- → `ContentUnavailableView` actions — retry button
- → `.task` — load on appear

MINIMUM OS

- `ProgressView`, `.redacted`: iOS 14
- `ContentUnavailableView`: iOS 17

UIKIT / REACT EQUIVALENT

- **UIKit** `UIActivityIndicatorView` + manual state view swapping
- **React Native** `isLoading` / `error` state + `ActivityIndicator`, React Query

⚠ COMMON MISTAKES

- Separate `isLoading`, `error` and `items` properties that disagree
- Showing an error alert when the user just navigated away (cancellation)
- Spinner covering data the user was reading during refresh

✓ TIPS & BEST PRACTICES

- Make sample placeholders the same size as real rows
- Use `.refreshable` for refresh; keep `.loaded` data during it
- Unit-test the model's phase transitions

QUICK SUMMARY

- enum for screen phases
- redacted skeletons while loading
- `ContentUnavailableView` for empty/failed

App Lifecycle & Scenes

APP & SCENE LIFECYCLE

`@Environment(\.scenePhase)` tells you when a scene is **active**, **inactive** or in the **background**, so you can save data or pause work. `onOpenURL` handles deep links and universal links. `@UIApplicationDelegateAdaptor` connects an `AppDelegate` for things SwiftUI doesn't cover yet, like push notification registration.

NOTES

- Read scenePhase in the App for app-wide reactions
- `.backgroundTask(.appRefresh("id"))` handles BGTasks
- `openWindow(id:)` opens extra windows on iPad/Mac
- `handlesExternalEvents` routes URLs to scenes (Mac)

LIFECYCLE HOOKS

```swift
final class AppDelegate: NSObject, UIApplicationDelegate {
    func application(_ app: UIApplication,
        didRegisterForRemoteNotificationsWithDeviceToken token: Data) {
        Push.register(token)
    }
}

@main
struct MyApp: App {
    @UIApplicationDelegateAdaptor(AppDelegate.self) var delegate
    @Environment(\.scenePhase) private var phase
    @State private var store = Store()

    var body: some Scene {
        WindowGroup {
            RootView()
                .environment(store)
                .onOpenURL { url in store.handle(url) }   // deep links
        }
        .onChange(of: phase) { _, new in
            if new == .background { store.save() }
        }
        .backgroundTask(.appRefresh("refresh")) { await store.refresh() }
    }
}
```

CODE EXPLAINED

- → `@UIApplicationDelegateAdaptor` — classic delegate hooks
- → `scenePhase` — active / inactive / background
- → `onChange(of: phase)` — save on background
- → `onOpenURL` — deep links
- → `.backgroundTask` — background refresh

MINIMUM OS

- `scenePhase`, `onOpenURL`, delegate adaptor: iOS 14
- `.backgroundTask`, `openWindow`: iOS 16

SCENE PHASES

PHASE	MEANING
`.active`	Foreground, receiving events
`.inactive`	Visible but not interactive
`.background`	Not visible; save now

UIKIT / REACT EQUIVALENT

- **UIKit** `AppDelegate` / `SceneDelegate` callbacks
- **React Native** `AppState.addEventListener('change')`, `Linking`

⚠ COMMON MISTAKES

- Doing heavy work when entering background: you get little time
- Reading scenePhase in a subview to mean the whole app (it's that scene)
- Handling deep links before the UI and models are ready

✓ TIPS & BEST PRACTICES

- Register the BGTask identifier in Info.plist (`BGTaskSchedulerPermittedIdentifiers`)
- Route URLs through your router (Sheet 31)
- SwiftData autosaves, but explicit saves on background are cheap insurance

QUICK SUMMARY

- `scenePhase` for foreground/background
- `onOpenURL` for links
- delegate adaptor for UIKit-only hooks

@AppStorage & @SceneStorage

SMALL PERSISTENT VALUES

`@AppStorage` is a property wrapper that reads and writes a **UserDefaults** key and redraws the view when it changes, which is perfect for settings. `@SceneStorage` saves small UI state **per scene** (selected tab, draft text) so it's restored when the system relaunches the app. Neither is for large data or secrets.

NOTES

- Supports Bool, Int, Double, String, URL, Data and String/Int RawRepresentable enums
- Pass `store: UserDefaults(suiteName:)` to share with widgets
- SceneStorage is per window and lost if the user force-quits
- Secrets belong in the Keychain (see Swift set)
- Swift set: O8O · UserDefaults & Keychain

@APPSTORAGE & @SCENESTORAGE

```
enum Theme: String, CaseIterable { case system, light, dark }

struct SettingsView: View {
    @AppStorage("theme") private var theme = Theme.system
    @AppStorage("fontScale") private var fontScale = 1.0
    @AppStorage("onboarded") private var onboarded = false

    var body: some View {
        Form {
            Picker("Theme", selection: $theme) {
                ForEach(Theme.allCases, id: \.self) { Text($0.rawValue) }
            }
            Slider(value: $fontScale, in: 0.8...1.4)
            Button("Replay Onboarding") { onboarded = false }
        }
    }
}

struct RootTabs: View {
    @SceneStorage("selectedTab") private var tab = "home"
    var body: some View { TabView(selection: $tab) { /* tabs */ } }
}
```

CODE EXPLAINED

- → `@AppStorage("key")` — UserDefaults-backed state
- → RawRepresentable enum — stored as its raw value
- → `$theme` — binding for controls
- → `@SceneStorage` — per-scene UI restoration

MINIMUM OS

- `@AppStorage`, `@SceneStorage`: iOS 14
- Shared App Group defaults: same API with `store:`

STORAGE OPTIONS

NEED	USE
User setting	@AppStorage
Restore UI on relaunch	@SceneStorage
Records, relationships	SwiftData
Tokens, passwords	Keychain
Files, images	FileManager

UIKIT / REACT EQUIVALENT

- **UIKit** `UserDefaults` + state restoration APIs
- **React Native** `AsyncStorage` / MMKV + a hook

⚠ COMMON MISTAKES

- Storing arrays of models in AppStorage via Data: use SwiftData or files
- Typos in duplicated key strings: centralize keys
- Expecting SceneStorage to survive a force-quit

✓ TIPS & BEST PRACTICES

- Define keys once: `enum Keys { static let theme = "theme" }`
- Use `.defaultAppStorage(_:)` to set the store for a subtree
- Apply the theme at the root with `.preferredColorScheme`

QUICK SUMMARY

- `@AppStorage` — settings
- `@SceneStorage` — UI restoration
- neither for big data or secrets

SwiftData Models

DEFINING SWIFTDATA MODELS

SwiftData is Apple's persistence framework built for SwiftUI. Mark a class with the `@Model` macro and its stored properties become persisted columns; relationships to other models are tracked automatically. Attach a **model container** to the app or a view with `.modelContainer(for:)`, and SwiftData creates the store and injects a context.

NOTES

- `@Model` classes are also Observable
- Every `@Model` needs an initializer
- `@Attribute(.unique)` enforces uniqueness (upsert on insert)
- Use an in-memory container for previews and tests
- Swift set: 096 · Macros

@MODEL & CONTAINER

```
import SwiftData

@Model final class Trip {
    @Attribute(.unique) var name: String
    var start: Date
    var notes = ""
    @Relationship(deleteRule: .cascade, inverse: \Stop.trip)
    var stops: [Stop] = []

    init(name: String, start: Date) { self.name = name; self.start = start }
}

@Model final class Stop {
    var place: String
    var trip: Trip?
    init(place: String) { self.place = place }
}

@main struct TravelApp: App {
    var body: some Scene {
        WindowGroup { TripList() }
            .modelContainer(for: Trip.self)       // Stop found via relation
    }
}

#Preview { TripList().modelContainer(for: Trip.self, inMemory: true) }
```

CODE EXPLAINED

- → `@Model` — persisted, observable class
- → `@Attribute(.unique)` — no duplicates
- → `@Relationship(deleteRule: .cascade)` — delete children too
- → `.modelContainer(for:)` — create store + context
- → `inMemory: true` — preview store

MINIMUM OS

- SwiftData: iOS 17 / macOS 14
- `#Unique`, `#Index`, history tracking: iOS 18

ATTRIBUTE OPTIONS

OPTION	EFFECT
`.unique`	One row per value
`.externalStorage`	Large data stored outside
`.transformable(by:)`	Custom encoding
`.ephemeral`	Not persisted
`@Transient`	Skip a property

UIKIT / REACT EQUIVALENT

- **UIKit** Core Data `NSManagedObject` + `NSPersistentContainer`
- **React Native** WatermelonDB / Realm / SQLite models

⚠ COMMON MISTAKES

- Forgetting the `init`: `@Model` classes don't get a memberwise init
- Using structs: `@Model` requires a class
- Changing models without a migration plan once users have data

✓ TIPS & BEST PRACTICES

- Use `VersionedSchema` + `SchemaMigrationPlan` for later changes
- Pass `ModelConfiguration(cloudKitDatabase: .automatic)` for iCloud sync
- Keep a `static var preview: ModelContainer` with sample data

QUICK SUMMARY

- `@Model final class` + `init`
- `@Relationship` / `@Attribute`
- `.modelContainer(for:)`

SwiftData Queries & Changes

QUERYING & CHANGING DATA

`@Query` fetches models into a view and **keeps the results live**: inserts, edits and deletes show up automatically. Filter with a `#Predicate`, sort with key paths or `SortDescriptors`. To change data, use the `modelContext` from the environment: `insert`, `delete`, and edit properties directly; changes autosave.

NOTES

- Predicates are type-checked Swift closures turned into queries
- Dynamic filters: build the `Query` in the view's `init`
- Autosave happens periodically; call `save()` for certainty
- Edit model properties with `@Bindable`
- Swift set: O94 · Key Paths

@QUERY & MODELCONTEXT

```
struct TripList: View {
    @Environment(\.modelContext) private var context
    @Query(sort: \Trip.start, order: .reverse) private var trips: [Trip]

    var body: some View {
        List {
            ForEach(trips) { TripRow(trip: $0) }
                .onDelete { for i in $0 { context.delete(trips[i]) } }
        }
        .toolbar {
            Button("Add", systemImage: "plus") {
                context.insert(Trip(name: "New Trip", start: .now))
            }
        }
    }
}

struct SearchResults: View {
    @Query private var trips: [Trip]
    init(text: String) {
        _trips = Query(filter: #Predicate<Trip> {
            text.isEmpty || $0.name.localizedStandardContains(text)
        }, sort: \.name)
    }
    var body: some View { List(trips) { Text($0.name) } }
}
```

CODE EXPLAINED

- → `@Query(sort:order:)` — live fetch
- → `context.insert` / `context.delete`
- → `#Predicate<Trip> { … }` — typed filter
- → `_trips = Query(…)` — dynamic query in init
- → edits to model properties — saved automatically

MINIMUM OS

- `@Query`, `#Predicate`, `ModelContext`: iOS 17
- `#Expression` in queries: iOS 18

CONTEXT API

CALL	DOES
`insert(model)`	Add
`delete(model)`	Remove
`try save()`	Write now
`try fetch(FetchDescriptor<T>())`	Fetch in code
`try fetchCount(_:)`	Count only

UIKIT / REACT EQUIVALENT

- **UIKit** `NSFetchedResultsController` + `NSManagedObjectContext`
- **React Native** WatermelonDB `observe()` queries / Realm live results

⚠ COMMON MISTAKES

- Unsupported functions in `#Predicate`: compile or runtime error; keep predicates simple
- Fetching in `body` with `context.fetch`: use `@Query`
- Deleting by offset from a filtered array that isn't the one shown

✓ TIPS & BEST PRACTICES

- Use `FetchDescriptor` with `fetchLimit` for background work
- Use `@ModelActor` for heavy imports off the main actor
- Debug with `-com.apple.CoreData.SQLDebug 1` launch argument

QUICK SUMMARY

- `@Query` — live results
- `modelContext` insert / delete
- `#Predicate` + sort

Animations

ANIMATING CHANGES

SwiftUI animates **state changes**. Wrap a change in `withAnimation` (explicit) to animate everything it affects, or attach `.animation(_:value:)` (implicit) to animate a view whenever one value changes. Springs are the default feel; `contentTransition` animates text and symbol changes in place.

NOTES

- Only animatable properties interpolate (opacity, frame, color, offset)
- Always pass `value:` to `.animation`
- `.spring(duration:bounce:)`, `.snappy`, `.bouncy` presets
- Respect Reduce Motion for large movement

EXPLICIT & IMPLICIT

```
@State private var expanded = false
@State private var count = 0

VStack {
    RoundedRectangle(cornerRadius: expanded ? 30 : 10)
        .fill(expanded ? .orange : .blue)
        .frame(width: expanded ? 240 : 120, height: 120)
        .animation(.spring(duration: 0.4, bounce: 0.3), value: expanded)

    Text("\(count)")
        .font(.largeTitle.monospacedDigit())
        .contentTransition(.numericText(value: Double(count)))

    Button("Toggle") { expanded.toggle() }
    Button("Add") {
        withAnimation(.snappy) { count += 1 }     // explicit
    }
}

withAnimation(.easeInOut(duration: 0.3)) { isOn = true } completion: {
    showNext = true
}
```

CODE EXPLAINED

- → `.animation(_:value:)` — implicit, tied to a value
- → `withAnimation { }` — explicit, around a change
- → `.spring(duration:bounce:)` — spring parameters
- → `contentTransition(.numericText())` — rolling numbers
- → `completion:` — run after the animation

MINIMUM OS

- `withAnimation`: iOS 13; `contentTransition`: iOS 16
- Spring presets, `completion:`: iOS 17

ANIMATION CURVES

ANIMATION	FEEL
`.default / .spring`	Natural spring
`.snappy`	Quick, small bounce
`.bouncy`	Playful bounce
`.smooth`	No bounce
`.easeInOut(duration:)`	Classic curve
`.linear`	Constant speed

UIKIT / REACT EQUIVALENT

- **`UIKit`** `UIView.animate(withDuration:)` / `UIViewPropertyAnimator`
- **`React Native`** `Animated` API or Reanimated `withSpring`

ACCESSIBILITY

- ♿ Check `@Environment(\.accessibilityReduceMotion)` and swap big motion for fades
- ♿ Don't animate essential info away before VoiceOver can read it

⚠ COMMON MISTAKES

- `.animation(.default)` without `value:`: deprecated, animates too much
- Animating layout of huge lists: janky; animate smaller pieces
- Expecting `if` insertions to animate without a transition (Sheet 44)

✓ TIPS & BEST PRACTICES

- Use `.transaction { $0.animation = nil }` to turn animation off for a change
- Use `.sensoryFeedback` alongside motion for physical feel
- Test with Slow Animations in the simulator's Debug menu

QUICK SUMMARY

- explicit `withAnimation`
- implicit `.animation(_:value:)`
- springs + content transitions

Transitions & Matched Geometry

INSERTION & HERO ANIMATIONS

A **transition** describes how a view enters and leaves when it's added or removed by an `if`. Combine built-in transitions or use `.asymmetric` for different in and out effects. `matchedGeometryEffect` morphs one view into another across a layout change, and the **zoom navigation transition** expands a tapped cell into the next screen.

NOTES

- Transitions need an animation (`withAnimation` around the change)
- `@Namespace` creates an ID space for matched geometry
- Only one view per id should be visible in matched geometry
- Zoom transition: mark source + destination with the same id

TRANSITIONS & MATCHED GEOMETRY

```
@State private var showBanner = false
@Namespace private var ns

if showBanner {
    Banner()
        .transition(.move(edge: .top).combined(with: .opacity))
}
Button("Show") { withAnimation { showBanner.toggle() } }

// morph a small badge into a large card
if expanded {
    Card().matchedGeometryEffect(id: "card", in: ns)
} else {
    Badge().matchedGeometryEffect(id: "card", in: ns)
}

// zoom from a grid cell into a detail screen
NavigationLink(value: photo) {
    Thumb(photo).matchedTransitionSource(id: photo.id, in: ns)
}
.navigationDestination(for: Photo.self) { p in
    PhotoDetail(p).navigationTransition(.zoom(sourceID: p.id, in: ns))
}
```

CODE EXPLAINED

- → `.transition(.move(edge:))` — enter/exit effect
- → `.combined(with:)` — mix transitions
- → `@Namespace` + `matchedGeometryEffect` — morph
- → `matchedTransitionSource` — zoom source
- → `.navigationTransition(.zoom)` — zoom push

MINIMUM OS

- Transitions: iOS 13; `matchedGeometryEffect` : iOS 14
- Zoom navigation transition: iOS 18

UIKIT / REACT EQUIVALENT

- **`UIKit`** Custom `UIViewControllerAnimatedTransitioning`
- **`React Native`** Reanimated layout animations / shared element transitions

BUILT-IN TRANSITIONS

TRANSITION	EFFECT
`.opacity`	Fade
`.scale`	Grow / shrink
`.slide`	Slide in leading, out trailing
`.move(edge:)`	Slide from an edge
`.push(from:)`	Push with fade
`.asymmetric(insertion:removal:)`	Different in/out

⚠ COMMON MISTAKES

- Toggling the `if` without `withAnimation`: transition doesn't play
- Two views with the same matched id visible at once: glitches
- Using opacity to hide a view and expecting a transition

✓ TIPS & BEST PRACTICES

- Wrap changing content in a container so transitions have a stable parent
- Use `.id(value)` + transition to animate content swaps
- Write custom transitions with the `Transition` protocol (iOS 17)

QUICK SUMMARY

- `.transition` for insert/remove
- `matchedGeometryEffect` to morph
- zoom transition for navigation

Phase & Keyframe Animators

MULTI-STEP ANIMATIONS

Phase animators cycle a view through a list of phases, animating between each, either continuously or each time a trigger changes. **Keyframe animators** give precise control: separate tracks for properties like scale and offset, each with timed keyframes. SF Symbols have their own built-in **symbol effects**.

NOTES

- Phase animator: simple sequences (pulse, shake)
- Keyframe animator: choreographed, timed motion
- Both re-run when `trigger:` changes
- Keyframe types: Linear, Spring, Cubic, Move

PHASE & KEYFRAME

```
Image(systemName: "heart.fill")
    .phaseAnimator([1.0, 1.3, 1.0], trigger: likes) { view, scale in
        view.scaleEffect(scale)
    } animation: { _ in .spring(duration: 0.2) }

struct Jump { var y = 0.0; var scale = 1.0 }

Text("🎉")
    .font(.system(size: 60))
    .keyframeAnimator(initialValue: Jump(), trigger: wins) { view, v in
        view.offset(y: v.y).scaleEffect(v.scale)
    } keyframes: { _ in
        KeyframeTrack(\.y) {
            SpringKeyframe(-60, duration: 0.25)
            SpringKeyframe(0, duration: 0.35)
        }
        KeyframeTrack(\.scale) {
            CubicKeyframe(1.4, duration: 0.2)
            CubicKeyframe(1.0, duration: 0.4)
        }
    }

Image(systemName: "wifi")
    .symbolEffect(.variableColor.iterative, isActive: searching)
```

CODE EXPLAINED

- → `.phaseAnimator(phases, trigger:)` — step through values
- → `animation:` — curve per phase
- → `.keyframeAnimator(initialValue:trigger:)` — tracks
- → `KeyframeTrack(\.y)` — one property's timeline
- → `.symbolEffect(isActive:)` — symbol animation

MINIMUM OS

- PhaseAnimator, KeyframeAnimator, symbol effects: iOS 17
- More symbol effects (`.wiggle` , `.breathe`): iOS 18

UIKIT / REACT EQUIVALENT

- **UIKit** `CAKeyframeAnimation` / `UIView.animateKeyframes`
- **React Native** Reanimated `withSequence` / Lottie

KEYFRAME TYPES

TYPE	MOTION
`LinearKeyframe`	Straight interpolation
`SpringKeyframe`	Spring into value
`CubicKeyframe`	Smooth curve
`MoveKeyframe`	Jump instantly

⚠ COMMON MISTAKES

- Phase animator without a trigger runs forever: fine for loaders, bad for content
- Keyframe tracks of different lengths: shorter ones hold their last value
- Using these for simple state changes `withAnimation` handles

✓ TIPS & BEST PRACTICES

- Store animation values in a small struct
- Pair celebratory animations with `.sensoryFeedback(.success, trigger:)`
- Respect Reduce Motion by skipping the trigger

QUICK SUMMARY

- phaseAnimator — sequences
- keyframeAnimator — timed tracks
- symbolEffect — animated symbols

Shapes, Paths & Canvas

DRAWING

Built-in shapes (`Rectangle`, `RoundedRectangle`, `Circle`, `Capsule`) can be filled, stroked and trimmed. Write your own `Shape` by returning a `Path` for a given rect. `Canvas` draws many items efficiently in immediate mode, and `TimelineView` redraws on a schedule for clocks and animations.

NOTES

- Shapes fill their frame by default
- `.trim(from:to:)` draws part of a stroke (progress rings)
- Canvas content isn't individual views: no per-item accessibility
- `Path` uses CGPoints in the given rect

SHAPES, PATHS, CANVAS

```
Circle()
    .trim(from: 0, to: progress)
    .stroke(.green, style: StrokeStyle(lineWidth: 12, lineCap: .round))
    .rotationEffect(.degrees(-90))
    .frame(width: 100, height: 100)

struct Triangle: Shape {
    func path(in r: CGRect) -> Path {
        Path { p in
            p.move(to: CGPoint(x: r.midX, y: r.minY))
            p.addLine(to: CGPoint(x: r.maxX, y: r.maxY))
            p.addLine(to: CGPoint(x: r.minX, y: r.maxY))
            p.closeSubpath()
        }
    }
}
Triangle().fill(.orange.gradient).frame(width: 80, height: 70)

TimelineView(.animation) { tl in
    Canvas { ctx, size in
        let t = tl.date.timeIntervalSinceReferenceDate
        for i in 0..<40 {
            let x = size.width * (Double(i) / 40)
            let y = size.height / 2 + sin(t * 2 + Double(i) / 3) * 30
            ctx.fill(Path(ellipseIn: CGRect(x: x, y: y, width: 6, height: 6)),
                     with: .color(.blue))
        }
    }
}
```

CODE EXPLAINED

- → `.trim` + `.stroke(style:)` — progress ring
- → `Shape` + `path(in:)` — custom shape
- → `.fill(.orange.gradient)` — quick gradient
- → `Canvas { ctx, size in }` — fast drawing
- → `TimelineView(.animation)` — per-frame redraw

MINIMUM OS

- Shapes, Path: iOS 13
- `Canvas`, `TimelineView`: iOS 15

UIKIT / REACT EQUIVALENT

- **`UIKit`** `draw(_:)` / `CAShapeLayer` / Core Graphics
- **`React Native`** react-native-svg / Skia

ACCESSIBILITY

- ♿ Describe charts and drawings with `.accessibilityLabel` or `.accessibilityRepresentation`
- ♿ Canvas is one element: add an `accessibilityChildren` description if needed

⚠ COMMON MISTAKES

- Hundreds of individual shape views in a ZStack: use Canvas
- Forgetting `-90°` rotation so the ring starts at the top
- Using `.stroke` and `.fill` together on a Shape (iOS 17+ allows chaining; earlier needs overlay)

✓ TIPS & BEST PRACTICES

- Use Swift Charts for real data visualization
- Make custom shapes `Animatable` via `animatableData`
- Use `.drawingGroup()` to flatten complex vector art into one layer

QUICK SUMMARY

- built-in shapes + trim/stroke
- custom `Shape` with `Path`
- `Canvas` + `TimelineView`

Custom Modifiers & Styles

REUSABLE STYLING

Package repeated modifier chains into a `ViewModifier` and expose it as a view extension, so `.cardStyle()` reads like a built-in modifier. For controls, write **styles**: a `ButtonStyle` gets the label and pressed state, and applying it with `.buttonStyle(...)` restyles every button in that subtree while keeping button behavior and accessibility.

NOTES

- ViewModifier: `func body(content: Content) -> some View`
- Styles exist for buttons, labels, toggles, pickers, progress, gauges
- Static extensions give dot syntax: `.buttonStyle(.primary)`
- Styles flow down the hierarchy like environment values
- Swift set: 070 · Protocol Extensions & POP

MODIFIER & BUTTONSTYLE

```
struct CardStyle: ViewModifier {
    func body(content: Content) -> some View {
        content
            .padding()
            .background(.background, in: .rect(cornerRadius: 16))
            .shadow(color: .black.opacity(0.1), radius: 8, y: 4)
    }
}
extension View { func cardStyle() -> some View { modifier(CardStyle()) } }

struct PrimaryButtonStyle: ButtonStyle {
    func makeBody(configuration: Configuration) -> some View {
        configuration.label
            .font(.headline)
            .padding(.horizontal, 24).padding(.vertical, 12)
            .background(.tint, in: .capsule)
            .foregroundStyle(.white)
            .scaleEffect(configuration.isPressed ? 0.96 : 1)
            .animation(.snappy, value: configuration.isPressed)
    }
}
extension ButtonStyle where Self == PrimaryButtonStyle {
    static var primary: Self { .init() }
}

VStack { Text("Hello").cardStyle(); Button("Go") {}.buttonStyle(.primary) }
```

CODE EXPLAINED

- → `ViewModifier` — reusable modifier chain
- → `extension View` — nice call syntax
- → `ButtonStyle.makeBody` — label + `isPressed`
- → `static var primary` — `.buttonStyle(.primary)`

MINIMUM OS

- `ViewModifier`, `ButtonStyle`: iOS 13
- `.background(_, in: shape)`: iOS 15

STYLE PROTOCOLS

PROTOCOL	APPLIED WITH
ButtonStyle	.buttonStyle(_:)
LabelStyle	.labelStyle(_:)
ToggleStyle	.toggleStyle(_:)
ProgressViewStyle	.progressViewStyle(_:)
TextFieldStyle	.textFieldStyle(_:)

UIKIT / REACT EQUIVALENT

- **UIKit** UIView subclasses or `UIButton.Configuration`; `UIAppearance`
- **React Native** Shared StyleSheet objects / styled components

⚠ COMMON MISTAKES

- Rebuilding a button from `onTapGesture` + modifiers instead of a ButtonStyle
- Hard-coded colors inside styles: use `.tint` and semantic colors
- Huge modifiers with many parameters: split them

✓ TIPS & BEST PRACTICES

- Read `@Environment(\.isEnabled)` in a style to draw disabled states
- Put your design system in a Swift package of modifiers and styles
- Use `PrimitiveButtonStyle` only when you need custom tap handling

QUICK SUMMARY

- `ViewModifier` + `extension View`
- `ButtonStyle` and friends
- static members for dot syntax

Dark Mode, Dynamic Type & Theming

APPEARANCE & TEXT SIZE

Apps must look right in **light and dark mode** and at every **Dynamic Type** size. Semantic colors and fonts handle most of it automatically. Read `colorScheme` and `dynamicTypeSize` from the environment for special cases, scale custom numbers with `@ScaledMetric`, and switch layouts at accessibility sizes.

NOTES

- Accessibility sizes go up to `.accessibility5`
- Custom fonts: `.custom(_:size:relativeTo:)` scales them
- `.preferredColorScheme` forces a scheme for a whole presentation
- Theme values travel well through the environment

ADAPTING TO SETTINGS

```
struct ProfileHeader: View {
    @Environment(\.colorScheme) private var scheme
    @Environment(\.dynamicTypeSize) private var typeSize
    @ScaledMetric(relativeTo: .body) private var avatar = 48.0

    var body: some View {
        let layout = typeSize.isAccessibilitySize
            ? AnyLayout(VStackLayout(alignment: .leading))
            : AnyLayout(HStackLayout())
        layout {
            Image("me").resizable().frame(width: avatar, height: avatar)
            Text("Ada").font(.custom("Avenir Next", size: 20, relativeTo: .title3))
        }
        .shadow(color: scheme == .dark ? .clear : .black.opacity(0.1), radius: 6)
    }
}

RootView()
    .preferredColorScheme(theme == .dark ? .dark : theme == .light ? .light : nil)
    .dynamicTypeSize(...DynamicTypeSize.accessibility3)   // cap if needed
```

CODE EXPLAINED

- → `colorScheme` — light / dark
- → `dynamicTypeSize.isAccessibilitySize` — switch layout
- → `@ScaledMetric` — numbers that scale with text
- → `.custom(_:size:relativeTo:)` — scalable custom font
- → `.preferredColorScheme(nil)` — follow system

MINIMUM OS

- `colorScheme` : iOS 13; `@ScaledMetric` : iOS 14
- `dynamicTypeSize` : iOS 15

UIKIT / REACT EQUIVALENT

- **UIKit** `traitCollection.userInterfaceStyle`, `UIFontMetrics`
- **React Native** `useColorScheme()`, `allowFontScaling`

PREVIEW CHECKS

MODIFIER	TESTS
`.preferredColorScheme(.dark)`	Dark mode
`.dynamicTypeSize(.accessibility5)`	Largest text
`.environment(\.legibilityWeight, .bold)`	Bold text
`.environment(\.colorSchemeContrast, .increased)`	High contrast

ACCESSIBILITY

- ♿ Test at the largest accessibility size; nothing should truncate important text
- ♿ Never block Dynamic Type entirely; cap only where layout truly breaks

⚠ COMMON MISTAKES

- Fixed font sizes that ignore Dynamic Type
- Images and icons that don't grow with text: use `@ScaledMetric`
- Shadows designed for light mode that look muddy in dark mode

✓ TIPS & BEST PRACTICES

- Use the Environment Overrides button in Xcode's debug bar
- Use Xcode Previews' variants mode to see all type sizes at once
- Keep a theme struct in the environment for brand values

QUICK SUMMARY

- semantic colors + fonts first
- `@ScaledMetric` for custom sizes
- switch layouts at accessibility sizes

Liquid Glass

APPLE'S 2025 DESIGN LANGUAGE

iOS 26 introduced **Liquid Glass**, a translucent material that reflects and refracts content behind it. Standard bars, tab bars, toolbars and sheets adopt it automatically when built with the iOS 26 SDK. For custom controls, `.glassEffect()` applies glass to any view, `GlassEffectContainer` blends nearby glass shapes, and `glassEffectID` morphs glass between states.

NOTES

- Rebuilding with the iOS 26 SDK updates standard controls for free
- Use glass for controls floating above content, not for content itself
- `.interactive()` makes glass react to touch
- Gate custom glass with `if #available(iOS 26, *)`

GLASS EFFECTS

```
@Namespace private var glass

GlassEffectContainer(spacing: 16) {             // shapes blend when close
    HStack(spacing: 16) {
        Button("Edit", systemImage: "pencil") { }
            .buttonStyle(.glass)
        if showMore {
            Button("Share", systemImage: "square.and.arrow.up") { }
                .buttonStyle(.glass)
                .glassEffectID("share", in: glass)
        }
    }
}

Text("Now Playing")
    .padding()
    .glassEffect(.regular.tint(.purple).interactive(), in: .capsule)

Button("Buy") { }.buttonStyle(.glassProminent)

Image("hero").resizable().scaledToFill()
    .backgroundExtensionEffect()                // extends under sidebar
```

CODE EXPLAINED

- → `.glassEffect(_:in:)` — glass behind a view
- → `.regular.tint().interactive()` — glass variants
- → `GlassEffectContainer` — merges nearby glass
- → `glassEffectID` — morph glass on change
- → `.buttonStyle(.glass / .glassProminent)` — glass buttons

MINIMUM OS

- Liquid Glass APIs: iOS 26 / macOS 26
- Older OS: fall back to `.background(.ultraThinMaterial)`

UIKIT / REACT EQUIVALENT

- **`UIKit`** `UIGlassEffect` in `UIVisualEffectView` (iOS 26)
- **`React Native`** No built-in equivalent; native modules or blur views

GLASS OPTIONS

API	EFFECT
`.regular`	Standard glass
`.clear`	More transparent
`.tint(color)`	Colored glass
`.interactive()`	Reacts to touch
`in: .capsule / .rect(cornerRadius:)`	Glass shape

⚠ COMMON MISTAKES

- Putting glass on large content areas: reduces legibility
- Stacking glass on glass
- Custom opaque nav bars fighting the new system bars

✓ TIPS & BEST PRACTICES

- Remove custom bar backgrounds and let the system draw glass
- Check legibility with Reduce Transparency turned on
- Use `ToolbarSpacer` to group toolbar items on iOS 26

QUICK SUMMARY

- `.glassEffect()` on floating controls
- `GlassEffectContainer` + `glassEffectID`
- iOS 26+, with material fallback

UIKit in SwiftUI

WRAPPING UIKIT VIEWS

When SwiftUI lacks a control, wrap the UIKit one. `UIViewRepresentable` (or `UIViewControllerRepresentable`) creates the UIKit object in `makeUIView`, updates it from SwiftUI state in `updateUIView`, and uses a **Coordinator** object as its delegate to send changes back through bindings.

NOTES

- `make…` runs once; `update…` runs on every relevant state change
- The Coordinator is created before `make` and lives as long as the view
- AppKit equivalents: `NSViewRepresentable`
- Check if a SwiftUI version exists first (Map, PhotosPicker, VideoPlayer)

UIVIEWREPRESENTABLE

```
struct RichTextView: UIViewRepresentable {
    @Binding var text: String

    func makeUIView(context: Context) -> UITextView {
        let tv = UITextView()
        tv.delegate = context.coordinator
        return tv
    }
    func updateUIView(_ tv: UITextView, context: Context) {
        if tv.text != text { tv.text = text }        // avoid loops
    }
    func makeCoordinator() -> Coordinator { Coordinator(text: $text) }

    final class Coordinator: NSObject, UITextViewDelegate {
        var text: Binding<String>
        init(text: Binding<String>) { self.text = text }
        func textViewDidChange(_ tv: UITextView) {
            text.wrappedValue = tv.text
        }
    }
}

struct SafariView: UIViewControllerRepresentable {
    let url: URL
    func makeUIViewController(context: Context) -> SFSafariViewController {
        SFSafariViewController(url: url)
    }
    func updateUIViewController(_ vc: SFSafariViewController,
                                context: Context) {}
}
```

CODE EXPLAINED

- → `makeUIView` — create once
- → `updateUIView` — push SwiftUI state in
- → `Coordinator` — delegate, sends changes out
- → `Binding` in coordinator — write back

MINIMUM OS

- Representables: iOS 13
- `sizeThatFits(_:uiView:context:)` for sizing: iOS 16

UIKIT / REACT EQUIVALENT

- **`UIKit`** This is the bridge into UIKit itself
- **`React Native`** Native UI components via `requireNativeComponent` / Fabric

ACCESSIBILITY

- ♿ UIKit views keep their own accessibility; set it on the UIKit object
- ♿ Make sure Dynamic Type works: use preferred fonts and `adjustsFontForContentSizeCategory`

REPRESENTABLE METHODS

METHOD	WHEN
`makeCoordinator()`	First, once
`makeUIView(context:)`	Once
`updateUIView(_:context:)`	State changes
`sizeThatFits(_:uiView:context:)`	Custom size

⚠ COMMON MISTAKES

- Setting values unconditionally in `updateUIView`: cursor jumps, loops
- Creating the UIKit view inside `updateUIView`
- Retaining the parent struct in the coordinator: copy the bindings you need

✓ TIPS & BEST PRACTICES

- Use `context.transaction.animation` to animate UIKit changes
- Keep representables small; put logic in a model

QUICK SUMMARY

- `make` once, `update` often
- Coordinator = delegate
- bindings carry changes back

SwiftUI in UIKit

HOSTING SWIFTUI IN UIKIT

Existing UIKit apps adopt SwiftUI screen by screen. `UIHostingController` wraps a SwiftUI view as a view controller you can push, present or embed as a child. `UIHostingConfiguration` puts SwiftUI content inside table and collection view cells. Share state through `@Observable` models that both sides hold.

NOTES

- Push/present a hosting controller like any VC
- Embedding as a child needs the standard containment calls
- `sizingOptions = .intrinsicContentSize` lets Auto Layout size it
- UIKit can call back into SwiftUI via closures passed into the view

UIHOSTINGCONTROLLER & CELLS

```
final class ProfileViewController: UIViewController {
    let model = ProfileModel()                          // @Observable

    func showEditor() {
        let editor = EditProfileView(profile: model) { [weak self] in
            self?.dismiss(animated: true)               // closure back to UIKit
        }
        present(UIHostingController(rootView: editor), animated: true)
    }

    override func viewDidLoad() {
        super.viewDidLoad()
        let host = UIHostingController(rootView: StatsCard(model: model))
        host.sizingOptions = .intrinsicContentSize
        addChild(host)
        view.addSubview(host.view)
        host.view.frame = view.bounds                   // or Auto Layout
        host.view.autoresizingMask = [.flexibleWidth, .flexibleHeight]
        host.didMove(toParent: self)
    }
}

cell.contentConfiguration = UIHostingConfiguration { TripRow(trip: trip) }
```

CODE EXPLAINED

- → `UIHostingController(rootView:)` — SwiftUI as a VC
- → `present` / push — like any VC
- → `addChild` + `didMove(toParent:)` — embed
- → `sizingOptions` — Auto Layout friendly sizing
- → `UIHostingConfiguration` — SwiftUI cells

MINIMUM OS

- `UIHostingController`: iOS 13
- `UIHostingConfiguration`, `sizingOptions`: iOS 16

UIKIT / REACT EQUIVALENT

- **`UIKit`** This is how UIKit hosts SwiftUI
- **`React Native`** Brownfield: `RCTRootView` inside a native app

MIGRATION PATH

STEP	APPROACH
New screens	SwiftUI in hosting controllers
Cells	UIHostingConfiguration
Shared state	@Observable models
Navigation	Keep UIKit until most screens move

⚠ COMMON MISTAKES

- Forgetting `addChild` / `didMove(toParent:)`: lifecycle and safe area bugs
- Rebuilding the hosting controller to update data: update the model instead
- Doubling navigation bars by wrapping hosted views in NavigationStack inside a UINavigationController

✓ TIPS & BEST PRACTICES

- Pass closures for actions UIKit should handle (dismiss, route)
- Use `@Observable` models so both UIKit and SwiftUI see changes
- Migrate leaf screens first, containers last

QUICK SUMMARY

- `UIHostingController` wraps views
- child containment for embedding
- `UIHostingConfiguration` for cells

Multiplatform SwiftUI

ONE CODEBASE, MANY PLATFORMS

Most SwiftUI code runs everywhere, but each platform has its own conventions: menus and windows on Mac, the Digital Crown on Watch, volumes and ornaments on Vision Pro. Use `#if os(...)` for platform-only code, prefer **semantic** APIs (toolbar placements, `Settings` scenes) that adapt by themselves, and design for pointer and keyboard where they exist.

NOTES

- One multiplatform target can build for iOS, iPadOS, macOS, visionOS
- Some modifiers are platform-only and fail to compile elsewhere
- Mac windows need sensible minimum sizes
- `.keyboardShortcut` helps Mac and iPad with keyboards

PLATFORM DIFFERENCES

```
@main struct NotesApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
                #if os(macOS)
                .frame(minWidth: 700, minHeight: 450)
                #endif
        }
        .commands {
            CommandMenu("Note") {
                Button("New Note") { newNote() }.keyboardShortcut("n")
            }
        }
        #if os(macOS)
        Settings { SettingsView() }
        MenuBarExtra("Quick Note", systemImage: "note") { QuickNote() }
        #endif
        #if os(visionOS)
        ImmersiveSpace(id: "focus") { FocusSpace() }
        #endif
    }
}
```

CODE EXPLAINED

- → `#if os(macOS)` — platform-only code
- → `.commands` / `CommandMenu` — Mac/iPad menus
- → `Settings` / `MenuBarExtra` — Mac scenes
- → `ImmersiveSpace` — visionOS

MINIMUM OS

- macOS SwiftUI apps: macOS 11+; `MenuBarExtra`: macOS 13
- visionOS: 1.0 (2024)

PLATFORM-SPECIFIC APIS

PLATFORM	EXAMPLES
macOS	Settings, MenuBarExtra, .commands
iOS	.navigationBarTitleDisplayMode, .keyboardType
watchOS	.digitalCrownRotation, NavigationStack-only UI
visionOS	ImmersiveSpace, .ornament, .glassBackgroundEffect
tvOS	.focusable, focus sections

UIKIT / REACT EQUIVALENT

- **`UIKit`** Mac Catalyst or separate AppKit code
- **`React Native`** `Platform.OS` checks; react-native-macos / windows

⚠ COMMON MISTAKES

- Sprinkling `#if os` everywhere: wrap differences in small helpers
- Shipping iPhone layouts stretched on Mac
- Ignoring hover and pointer: add `.hoverEffect` / `.onHover`

✓ TIPS & BEST PRACTICES

- Share models and most views; specialize only top-level layout
- Use `#if canImport(UIKit)` for UIKit-dependent helpers
- Test keyboard navigation on iPad and Mac

QUICK SUMMARY

- `#if os()` for platform code
- semantic APIs adapt automatically
- Mac: commands, Settings, windows

Accessibility

ACCESSIBLE SWIFTUI

Standard controls are accessible by default: VoiceOver reads labels and values, and Dynamic Type scales text. Your job is to fill gaps: **labels** for icon-only views, **traits** for custom controls, **grouping** of related elements, **actions** for gestures, and **hiding** decoration. Verify with VoiceOver and Xcode's Accessibility Inspector.

NOTES

- Label = what it is; value = its state; hint = what happens
- `.combine` merges children into one element
- `@AccessibilityFocusState` moves VoiceOver focus
- Accessibility Inspector: Xcode › Open Developer Tool

ACCESSIBILITY MODIFIERS

```
HStack {
    Image(systemName: "star.fill").foregroundStyle(.yellow)
    Text("4.8")
    Text("(2,104 reviews)").foregroundStyle(.secondary)
}
.accessibilityElement(children: .combine)        // one element
.accessibilityLabel("Rated 4.8 stars from 2,104 reviews")

Button(action: toggleFavorite) {
    Image(systemName: isFav ? "heart.fill" : "heart")
}
.accessibilityLabel("Favorite")
.accessibilityValue(isFav ? "On" : "Off")
.accessibilityHint("Saves this recipe")

Text("Ingredients").font(.title2)
    .accessibilityAddTraits(.isHeader)            // rotor heading

SwipeCard(item)
    .accessibilityAction(named: "Archive") { archive(item) }

Image("divider").accessibilityHidden(true)       // decorative

CustomSlider(value: $v)
    .accessibilityRepresentation { Slider(value: $v) }
```

CODE EXPLAINED

- → `children: .combine` — group a row
- → label / value / hint — describe a control
- → `.isHeader` trait — headings for the rotor
- → `accessibilityAction(named:)` — gesture alternative
- → `accessibilityRepresentation` — act like a standard control

MINIMUM OS

- Accessibility modifiers: iOS 14–15
- `accessibilityRepresentation`, `@AccessibilityFocusState`: iOS 15

UIKIT / REACT EQUIVALENT

- **UIKit** `accessibilityLabel`, `accessibilityTraits`, `UIAccessibilityCustomAction`
- **React Native** `accessibilityLabel`, `accessibilityRole`, `accessibilityActions`

TESTING CHECKLIST

CHECK	HOW
Every control has a label	VoiceOver / Inspector audit
Largest text size	Dynamic Type preview
Reduce Motion	Settings › Accessibility
Contrast	Increase Contrast + Inspector
Keyboard / Switch Control	Full Keyboard Access

⚠ COMMON MISTAKES

- Labels that repeat the trait: "Favorite button" (VoiceOver already says "button")
- Combining elements that contain separate buttons: they become unreachable
- Custom controls built from shapes with no label or value

✓ TIPS & BEST PRACTICES

- Run the Accessibility Inspector's Audit on every screen
- Use `.accessibilitySortPriority` to fix reading order
- Write UI tests against accessibility labels; it doubles as a check

QUICK SUMMARY

- label / value / hint / traits
- combine, hide, add actions
- test with VoiceOver + Inspector

Localization

TRANSLATING YOUR APP

String literals in `Text`, `Button` and other views are `LocalizedStringKey`s, looked up automatically. Xcode **String Catalogs** (`.xcstrings`) collect every string at build time, track translation state, and handle **plural** and device variations. Use `String(localized:)` for strings created in code, and format numbers and dates with FormatStyle so they localize too.

NOTES

- Interpolations become format arguments: `"\(count) items"`
- Mark plural variants in the catalog ("Vary by Plural")
- Add comments for translators with `comment:`
- Right-to-left languages flip layouts automatically
- Swift set: O17 · String Interpolation & Formatting

LOCALIZED STRINGS

```swift
Text("Welcome")                                  // looked up
Text("\(count) photos")                          // plural variants in catalog
Button("Delete All") { }

let title = String(localized: "Settings",
                   comment: "Navigation title for settings screen")
let msg = String(localized: "Saved \(count) items")

Text(verbatim: user.name)                        // never translated
Text(LocalizedStringResource("onboarding.title"))  // keyed resource

Text(price, format: .currency(code: currency))     // locale-aware
Text(date, format: .dateTime.month().day())

#Preview("German") {
    SettingsView().environment(\.locale, Locale(identifier: "de"))
}
#Preview("RTL") {
    SettingsView().environment(\.layoutDirection, .rightToLeft)
}
```

CODE EXPLAINED

- → string literal in Text — localized key
- → interpolation — becomes a plural-capable format
- → `String(localized:comment:)` — strings in code
- → `Text(verbatim:)` — skip lookup
- → `\.locale` / `\.layoutDirection` — preview languages

MINIMUM OS

- String Catalogs: Xcode 15
- `String(localized:)`, `LocalizedStringResource`: iOS 15–16

UIKIT / REACT EQUIVALENT

- **`UIKit`** `NSLocalizedString` + `.strings` / `.stringsdict` files
- **`React Native`** i18next / react-intl translation keys

CATALOG STATES

STATE	MEANING
`New`	Needs translation
`Translated`	Done
`Needs Review`	Source changed
`Stale`	No longer in code

⚠ COMMON MISTAKES

- Building sentences by concatenation: word order differs by language
- Passing a String variable to `Text` and expecting translation: it's displayed verbatim
- Hard-coded left/right padding: use leading/trailing

✓ TIPS & BEST PRACTICES

- Export with Product › Export Localizations for translators
- Use pseudo-languages (double length, RTL) in scheme options
- Keep keys natural-language; catalogs handle the rest

QUICK SUMMARY

- literals are localized keys
- String Catalogs + plurals
- `String(localized:)` in code

Performance & Debugging

MAKING VIEWS FAST

SwiftUI performance comes down to **how often** `body` runs and **how much** each run does. Keep `body` cheap, keep view identity stable, read only the state each view needs (Observation helps), and use lazy containers for long content. `Self._printChanges()` explains why a view redrew, and Instruments shows slow bodies and update causes.

NOTES

- Unstable IDs (like `UUID()` in body) rebuild views every time
- `AnyView` hides structure from the diffing engine
- Pass child views only the data they use
- Format and sort in the model, not in body

DIAGNOSING UPDATES

```
struct MessageRow: View {
    let message: Message
    var body: some View {
        let _ = Self._printChanges()          // logs why body ran
        HStack { Text(message.sender).bold(); Text(message.preview) }
    }
}

// ✗ new identity every render → state lost, slow
ForEach(items, id: \.self) { … }              // when items aren't unique
List(items) { Row(id: UUID(), item: $0) }     // never generate IDs in body

// ✓ precompute in the model
@Observable final class Inbox {
    var messages: [Message] = [] { didSet { sorted = messages.sorted(by: >) } }
    private(set) var sorted: [Message] = []
}

struct Avatar: View, Equatable {              // skip redraws if equal
    let url: URL
    var body: some View { AsyncImage(url: url) }
}
Avatar(url: u).equatable()
```

CODE EXPLAINED

- → `Self._printChanges()` — why did body run?
- → stable IDs — keep identity and state
- → precompute — keep body cheap
- → `Equatable` + `.equatable()` — skip unchanged views

MINIMUM OS

- `_printChanges`: iOS 15 (debug only)
- Observation's per-property tracking: iOS 17

INSTRUMENTS & TOOLS

TOOL	SHOWS
SwiftUI instrument	Body updates and their causes
Time Profiler	Hot functions
Hangs	Main-thread stalls
View Hierarchy Debugger	Layout
_printChanges()	Changed dependencies

UIKIT / REACT EQUIVALENT

- **UIKit** Avoiding `reloadData`; profiling `layoutSubviews`
- **React Native** `React.memo`, `useMemo`, FlatList `keyExtractor`

⚠ COMMON MISTAKES

- Sorting, filtering or formatting large data inside `body`
- Whole-screen models read by every row: rows redraw on any change
- `.id(UUID())` to force refresh: destroys and rebuilds the view

✓ TIPS & BEST PRACTICES

- Split big views so changes redraw only small pieces
- Profile release builds on a real device
- Use `LazyVStack` / `List` for anything over ~50 items

QUICK SUMMARY

- cheap bodies, stable identity
- read only needed state
- `_printChanges` + Instruments

App Architecture

STRUCTURING AN APP

A common SwiftUI architecture: **views** stay thin and declarative; **@Observable models** (view models or feature stores) hold state and logic; **services** (API clients, persistence) sit behind protocols and are injected, often through the environment. This keeps views previewable and models unit-testable without a UI.

NOTES

- One model per screen or feature, not per view
- Services as protocols allow fakes in previews and tests
- Inject dependencies at the top, not as singletons
- Feature modules can be local Swift packages
- Swift set: 069 · Protocols — Basics, 102 · Sendable & Strict Concurrency

MODEL + INJECTED SERVICE

```
protocol RecipeService: Sendable {
    func recipes() async throws -> [Recipe]
}
struct LiveRecipeService: RecipeService { /* URLSession */ }
struct MockRecipeService: RecipeService {
    func recipes() async throws -> [Recipe] { Recipe.samples }
}

extension EnvironmentValues {
    @Entry var recipeService: any RecipeService = LiveRecipeService()
}

@MainActor @Observable final class RecipeListModel {
    private let service: any RecipeService
    var recipes: [Recipe] = []
    init(service: any RecipeService) { self.service = service }
    func load() async { recipes = (try? await service.recipes()) ?? [] }
}

struct RecipeListView: View {
    @Environment(\.recipeService) private var service
    @State private var model: RecipeListModel?
    var body: some View {
        List(model?.recipes ?? []) { Text($0.name) }
            .task {
                if model == nil { model = RecipeListModel(service: service) }
                await model?.load()
            }
    }
}

#Preview { RecipeListView().environment(\.recipeService, MockRecipeService()) }
```

CODE EXPLAINED

- → service protocol — swap live/mock
- → `@Entry` — inject through the environment
- → `@MainActor @Observable` model — state + logic
- → model created from env in `.task`
- → preview injects a mock

MINIMUM OS

- `@Observable`: iOS 17; `@Entry`: Xcode 16
- Pattern works on iOS 16 with ObservableObject

UIKIT / REACT EQUIVALENT

- **`UIKit`** MVVM / Coordinator with view models
- **`React Native`** Hooks + context providers; Redux Toolkit slices

⚠ COMMON MISTAKES

- Networking code directly in views
- Global singletons that previews and tests can't replace
- A view model for every tiny view

✓ TIPS & BEST PRACTICES

- Pass dependencies into model inits so tests can construct them
- Start with plain `@State` in views; add a model when logic grows
- Keep models free of SwiftUI imports where possible

QUICK SUMMARY

- thin views, `@Observable` models
- services behind protocols
- inject via environment / init

Testing SwiftUI Apps

TESTING SWIFTUI APPS

Test **logic** in models with fast unit tests (Swift Testing), using fake services. Test **flows** with XCUITest, which drives the real app through accessibility; give important elements an `accessibilityIdentifier`. Previews catch visual regressions during development, and snapshot testing libraries compare rendered images in CI.

NOTES

- Models with injected services are easy to unit test
- Mark model tests `@MainActor` when the model is
- Pass launch arguments to switch the app to mock data
- Snapshot testing: e.g. pointfreeco/swift-snapshot-testing
- Swift set: O87 · Swift Testing

UNIT & UI TESTS

```
import Testing
@testable import Recipes

@MainActor
@Test func loadsRecipes() async {
    let model = RecipeListModel(service: MockRecipeService())
    await model.load()
    #expect(model.recipes.count == Recipe.samples.count)
}

// UI test target (XCTest)
import XCTest
final class RecipeUITests: XCTestCase {
    func testAddRecipe() {
        let app = XCUIApplication()
        app.launchArguments = ["-useMockData"]
        app.launch()
        app.buttons["addRecipe"].tap()
        app.textFields["recipeName"].typeText("Soup")
        app.buttons["Save"].tap()
        XCTAssertTrue(app.staticTexts["Soup"].waitForExistence(timeout: 2))
    }
}

// In the app:
Button("Add", systemImage: "plus", action: add)
    .accessibilityIdentifier("addRecipe")
```

CODE EXPLAINED

- → `@MainActor @Test` — unit test a model
- → mock service — no network in tests
- → `XCUIApplication` + launch arguments — UI test
- → `accessibilityIdentifier` — stable UI handle
- → `waitForExistence` — handle async UI

MINIMUM OS

- Swift Testing: Xcode 16
- XCUITest: all supported OS versions

UIKIT / REACT EQUIVALENT

- **`UIKit`** Same: XCTest + XCUITest; view controllers harder to unit test
- **`React Native`** Jest + React Native Testing Library; Detox / Maestro for E2E

TEST TYPES

TYPE	CATCHES
Model unit tests	Logic, state transitions
UI tests	Broken flows
Snapshot tests	Visual regressions
Previews	Quick visual checks

⚠ COMMON MISTAKES

- Testing view internals: test models and user-visible behavior instead
- UI tests depending on real network data: flaky
- Locating elements by display text that gets localized

✓ TIPS & BEST PRACTICES

- Read `ProcessInfo.processInfo.arguments` at launch to inject mocks
- Keep UI tests few and focused on critical flows
- Run tests on CI with `xcodebuild test -scheme App -destination …`

QUICK SUMMARY

- unit-test `@Observable` models
- XCUITest via accessibility identifiers
- snapshots for visuals

Quick Reference — Views & Layout

VIEWS

VIEW	USE
`Text("…")`	Text
`Image(systemName:)`	Symbol
`Label(_:systemImage:)`	Text + icon
`Button(_:) { }`	Action
`AsyncImage(url:)`	Remote image
`ProgressView()`	Spinner / bar

LAYOUT

VIEW	USE
`VStack / HStack`	Column / row
`ZStack`	Layers
`Spacer()`	Push apart
`Grid / GridRow`	Aligned table
`LazyVGrid(columns:)`	Scrolling grid
`ViewThatFits`	Fallback layouts

SIZING

MODIFIER	EFFECT
`.padding()`	Space around
`.frame(width:height:)`	Fixed size
`.frame(maxWidth: .infinity)`	Fill width
`.fixedSize()`	No truncation
`.layoutPriority(1)`	Wins space
`.ignoresSafeArea()`	Edge to edge

APPEARANCE

MODIFIER	EFFECT
`.font(.headline)`	Semantic font
`.foregroundStyle(.secondary)`	Text color
`.background(.thinMaterial, in: .capsule)`	Shaped background
`.overlay(alignment:) { }`	Decorate
`.clipShape(.rect(cornerRadius:))`	Clip
`.tint(.green)`	Control accent

CONTROLS

CONTROL	BINDS TO
`TextField(_:text:)`	String
`Toggle(_:isOn:)`	Bool
`Slider(value:in:)`	Double
`Stepper(_:value:in:)`	Int
`Picker(_:selection:)`	Tagged value
`DatePicker(_:selection:)`	Date

LISTS & SCROLLING

API	USE
`List(items) { }`	Rows
`ForEach(items) { }`	Dynamic content
`.onDelete / .onMove`	Editing
`.swipeActions { }`	Swipe buttons
`ScrollView + LazyVStack`	Custom scroll
`.searchable(text:)`	Search

DRAWING

API	USE
`Circle / Capsule`	Shapes
`.trim(from:to:)`	Partial stroke
`Path { }`	Custom path
`Canvas { ctx, size in }`	Fast drawing
`.glassEffect()`	Liquid Glass (26)

⚠ COMMON MISTAKES

- Modifier order changes results: padding before background
- Lazy grids and stacks must sit in a ScrollView
- Fixed font sizes break Dynamic Type

✓ TIPS & BEST PRACTICES

- Debug layout with `.border(.red)`
- Prefer semantic fonts, colors and placements
- Extract subviews early

QUICK SUMMARY

- views + modifiers
- stacks, grids, frames
- controls bind to state

Quick Reference — State & Navigation

PROPERTY WRAPPERS

WRAPPER	USE
`@State`	View-owned value/model
`@Binding`	Borrowed writable value
`@Bindable`	Bindings into a model
`@Environment(\.key)`	System value
`@Environment(T.self)`	Injected model
`@AppStorage("k")`	UserDefaults
`@FocusState`	Keyboard focus
`@Query`	SwiftData fetch

DATA FLOW

SITUATION	PATTERN
`Local toggle`	@State
`Child edits parent value`	@Binding
`Shared logic + state`	@Observable class
`App-wide model`	.environment(model)
`Edit model in form`	@Bindable
`Persisted records`	@Model + @Query

NAVIGATION

API	USE
`NavigationStack(path:)`	Push stack
`NavigationLink(value:)`	Push data
`.navigationDestination(for:)`	Type → view
`TabView { Tab(…) }`	Tabs
`NavigationSplitView`	Columns
`.navigationTitle(_:)`	Title

PRESENTATION

API	USE
`.sheet(item:)`	Modal with data
`.fullScreenCover`	Full screen
`.alert`	Message
`.confirmationDialog`	Choices
`.popover`	Anchored bubble
`@Environment(\.dismiss)`	Close

LIFECYCLE & ASYNC

API	USE
`.task { }`	Load, auto-cancel
`.task(id:)`	Reload on change
`.onChange(of:)`	React to value
`.refreshable { }`	Pull to refresh
`scenePhase`	Foreground/background

ANIMATION

API	USE
`withAnimation { }`	Explicit
`.animation(_:value:)`	Implicit
`.transition(_:)`	Insert/remove
`matchedGeometryEffect`	Morph
`.phaseAnimator`	Sequences

⚠ COMMON MISTAKES

- `@State` initialized from init params ignores later changes
- Nested NavigationStacks break pushing
- Bool sheets with separate item state show stale data

✓ TIPS & BEST PRACTICES

- One owner per model; others borrow
- Navigation state lives in a path array
- Use `.task` instead of `Task {}` in onAppear

QUICK SUMMARY

- wrappers by ownership
- stack / tabs / split navigation
- task, onChange, animations